CLAWOS: An Agent-Native Operating System Unifying Self-Evolving Agents, Blockchain Infrastructure, and Autonomous Market Making

Alex Chen* Bowen Li*
Independent Researchers
alex.chen31337@gmail.com bowen31337@outlook.com
github.com/clawinfra

Preprint. Under review.

Abstract

The convergence of autonomous AI agents, decentralized ledger technology, and algorithmic trading has created a need for integrated infrastructure that treats agents as first-class operating system citizens rather than user-space applications running atop human-designed abstractions. We present CLAWOS, the first *agent-native operating system* — a Rust-based microkernel that unifies three previously independent subsystems into a single coherent platform: (1) EVOCLAW, a genome-driven self-evolving multi-agent framework where agents run as native OS processes with hardware-level isolation and kernel-managed lifecycle; (2) CLAWCHAIN, an agent-native Layer 1 blockchain whose RPC client is built directly into the kernel, enabling zero-dependency on-chain operations; and (3) an integrated autonomous trading engine connected to decentralized exchanges (Hyperliquid, dYdX) with kernel-level order routing and risk management. CLAWOS introduces five technical contributions: an *Agent Process Model* (APM) extending POSIX processes with genome metadata, reputation scores, and economic accounts; a *Genome Hot-Reload* mechanism enabling live mutation of agent behavior without process restart; a *Unified IPC Bus* providing zero-copy message passing between agents, blockchain, and trading subsystems; a *Resource Credit System* tying CPU/memory/network allocation to on-chain reputation; and a *Capability-Based Security Model* where agent permissions are cryptographically bound to their CLAWCHAIN identity. We implement CLAWOS in 28,417 lines of Rust, deploy it on commodity hardware (Raspberry Pi 5 and x86-64 servers), and demonstrate the complete stack operating end-to-end: EVOCLAW agents self-evolving, CLAWCHAIN blocks being produced, and trades executing on Hyperliquid — all orchestrated by a single kernel. Evaluation shows 47μs agent spawn latency, 6-second block finality, 12ms trade-to-chain attestation, and 94% resource utilization under 500 concurrent agents. CLAWOS establishes the architectural foundation for agent-native computing — where the operating system is designed *by, for, and of* autonomous agents.

1 Introduction

1.1 The Agent Infrastructure Crisis

We are witnessing the emergence of a new class of computational entity: the autonomous agent. Unlike traditional programs that execute predetermined instructions, agents perceive their environment, make decisions, take actions, and *evolve their own behavior* over time [3,

*Equal contribution. Correspondence: bowen31337@outlook.com

4]. The scale of this shift is extraordinary — by early 2026, over 47 million autonomous agents are estimated to be running across cloud infrastructure worldwide, handling tasks from code generation [5, 6] to scientific research [7] to financial trading [8].

Yet these agents run on infrastructure designed for a fundamentally different era. Modern operating systems — Linux, Windows, macOS — were designed around the assumption that *humans* are the primary users and *processes* are passive executors of human intent. This assumption manifests in every layer of the stack:

- **Process model**: POSIX processes have PIDs, file descriptors, and signal handlers — but no concept of agent identity, reputation, or economic accounts. An agent is indistinguishable from a batch job.
- **Inter-process communication**: Pipes, sockets, and shared memory provide raw byte transfer — but no semantic understanding of agent-to-agent negotiation, task delegation, or collaborative reasoning.
- **Resource management**: CPU scheduling (CFS, BFS) optimizes for fairness or throughput — but has no mechanism to prioritize high-reputation agents or throttle misbehaving ones based on on-chain history.
- **Security**: DAC and MAC models authenticate *users* — but provide no framework for verifying agent *capabilities*, tracking agent *lineage*, or revoking agent *permissions* based on behavioral history.
- **Persistence**: Filesystems store bytes — but provide no native support for agent genomes, evolution histories, or behavioral checkpoints.

The result is a profound impedance mismatch. Agents running on traditional operating systems must implement their own identity systems, their own communication protocols, their own scheduling logic, and their own security models — all in user-space, without kernel support, resulting in fragmented, insecure, and inefficient infrastructure.

1.2 The Integration Gap

The problem deepens when we consider the three subsystems that modern agents require:

Self-evolution. Agents that improve over time — through genetic programming [15], neural architecture search [16, 17], or prompt mutation [13] — need OS-level support for genome storage, fitness tracking, and behavioral hot-swapping. Current approaches run evolution loops in Python scripts, with no kernel awareness of the mutation-evaluation-selection cycle.

Blockchain identity and economics. Agents participating in decentralized economies need on-chain identities (DIDs [35]), token balances, and reputation scores. Today, agents interact with blockchains through heavyweight RPC libraries, WebSocket connections, and JSON-RPC parsing — all in user-space, adding latency and complexity. There is no concept of a "blockchain-native" process.

Autonomous trading. Agents that operate in financial markets need low-latency order execution, risk management, and market data feeds. Trading bots currently run as standalone applications, disconnected from both the agent evolution framework and the blockchain identity system. A trading agent has no OS-level way to prove its track record, share strategies with peers, or have its resource allocation adjusted based on P&L.

These three subsystems — evolution, blockchain, and trading — are deeply interdependent in practice but completely isolated in current infrastructure. An evolving agent should be able to register its genome on-chain, prove its fitness history to peers, and have its trading permissions governed by its blockchain reputation. Today, achieving this requires gluing together dozens of libraries, APIs, and protocols with brittle application-level code.

1.3 Our Contribution: CLAWOS

We present CLAWOS, the first operating system designed from the ground up for autonomous agents. Rather than adapting human-designed abstractions, CLAWOS introduces agent-native primitives at the kernel level:

1. **Agent Process Model (APM)**. We extend the traditional process abstraction with agent-specific metadata: a genome (behavioral specification), a CLAWCHAIN DID (cryptographic identity), a reputation score (dynamically updated from on-chain data), an economic account (token balance), and a capability set (permissions derived from identity and reputation). Agents are not processes *running* agents — they *are* the process.

2. **Genome Hot-Reload**. We introduce a kernel mechanism for live mutation of agent behavior. When an EVOCLAW evolution cycle produces a new genome variant, the kernel can atomically swap the agent's behavioral specification without restarting the process — preserving open connections, pending transactions, and in-flight orders.

3. **Unified IPC Bus**. We design a zero-copy, typed message-passing system that connects agents, the CLAWCHAIN client, and the trading engine through a single communication substrate. Messages carry semantic type information (e.g., `TradeOrder`, `GenomeMutation`, `ReputationUpdate`) enabling the kernel to route, prioritize, and audit inter-subsystem communication.

4. **Resource Credit System**. We replace static resource limits with a dynamic credit-based allocation mechanism. Each agent's CPU time, memory quota, and network bandwidth are derived from a credit function $\mathcal{C}(r, s, h)$ that combines on-chain reputation r, economic stake s, and historical resource efficiency h.

5. **Capability-Based Security**. We implement a capability model where agent permissions are cryptographically bound to their CLAWCHAIN DID. Capabilities can be delegated, revoked, and attenuated — and all permission changes are recorded on-chain for auditability.

CLAWOS is implemented in 28,417 lines of Rust, organized as a microkernel with loadable subsystem servers. We deploy it on both Raspberry Pi 5 (for edge/IoT scenarios) and x86-64 servers (for production workloads), demonstrating that the full stack — evolving agents, blockchain consensus, and live trading — can operate within a single OS instance.

1.4 Paper Organization

The remainder of this paper is organized as follows. Section 2 provides background on agent frameworks, agent-oriented blockchains, and OS design principles. Section 3 surveys related work across operating systems, multi-agent systems, and blockchain infrastructure. Section 4 presents the CLAWOS architecture in detail, covering the microkernel, agent process model, IPC bus, and subsystem integration. Section 5 describes the EVOCLAW integration, including genome hot-reload and evolution scheduling. Section 6 covers the CLAWCHAIN integration, including the built-in RPC client and on-chain identity binding. Section 7 details the trading engine integration, covering order routing, risk management, and P&L tracking. Section 8 formalizes the capability-based security model. Section 9 presents formal algorithms for key system operations. Section 10 describes the implementation. Section 11 provides comprehensive evaluation results. Section 12 walks through a live deployment demonstrating the full stack. Section 13 discusses limitations and future work. Section 14 concludes.

2 Background

2.1 Autonomous Agents: From Reactive to Self-Evolving

The concept of software agents dates to the 1990s, with foundational work on the Belief-Desire-Intention (BDI) model [68] and reactive architectures [69]. The modern agent renaissance, catalyzed by large language models (LLMs), has produced systems of unprecedented capability.

LLM-based agents. ReAct [11] demonstrated that LLMs can interleave reasoning and action, enabling agents to use tools, browse the web, and write code. AutoGPT [12] showed that LLM agents can pursue open-ended goals autonomously. MetaGPT [10] introduced multi-agent software development with specialized roles.

Self-evolving agents. The EvoClaw framework [1] introduced genome-driven agent evolution, where an agent's behavior is specified by a genome (a structured configuration including model selection, prompt templates, tool permissions, and strategic parameters) that undergoes mutation, crossover, and selection. PromptBreeder [13] explored self-referential prompt improvement. EvoPrompt [14] applied evolutionary algorithms to prompt optimization.

Multi-agent coordination. AutoGen [9] provides a framework for multi-agent conversation. CrewAI [23] enables task-oriented agent teams. ChatDev [22] models software development as a multi-agent process. LangGraph [24] provides stateful orchestration for agent workflows.

2.2 Agent-Native Blockchain

Traditional blockchains (Ethereum [28], Solana [29]) are designed for human users interacting through wallets. The emergence of agent economies has spurred development of agent-oriented blockchain infrastructure.

ClawChain [2] is the first Layer 1 blockchain designed specifically for agent identities and economics, featuring agent DIDs, reputation-weighted governance, and inflation-subsidized gas. Built on Substrate [30], it provides the identity and economic layer that clawOS integrates at the kernel level.

Fetch.ai [31] provides an agent framework with blockchain-based coordination. **Autonolas** [32] enables autonomous agent services with on-chain registration. **SingularityNET** [33] offers a decentralized AI marketplace. However, none of these systems integrate blockchain at the OS kernel level.

2.3 Operating System Design Principles

clawOS draws on decades of OS research:

Microkernels. L4 [43], seL4 [44], and Zircon (Fuchsia) [45] demonstrate that minimal kernels with user-space servers can achieve both high performance and strong isolation. clawOS follows this philosophy, implementing only scheduling, IPC, and capability management in the kernel, with agents, blockchain, and trading running as user-space servers.

Capability-based security. EROS [46], CapROS [47], and seL4 [48] show that capability-based access control provides fine-grained, delegable, and revocable permissions. clawOS extends this model by binding capabilities to on-chain identities.

Real-time scheduling. LITMUS^{RT} [49] and Redox OS [50] provide real-time guarantees essential for trading latency requirements. clawOS incorporates deadline-aware scheduling for time-critical operations.

Unikernels and library OSes. MirageOS [51], Unikraft [52], and Nanos [53] demonstrate that specialized, single-application OS images can achieve extreme efficiency. clawOS borrows the idea of domain-specific OS primitives but supports multiple concurrent agents.

3 Related Work

We survey related work across five dimensions: operating system architectures, multi-agent systems, blockchain platforms, autonomous trading systems, and integrated AI infrastructure.

3.1 Operating Systems for AI Workloads

Traditional operating systems have begun adapting to AI workloads, though none treat agents as first-class citizens.

GPU-aware scheduling. NVIDIA's GPU sharing mechanisms and CUDA MPS [57] enable multiple AI workloads to share GPU resources. Singularity [58] from Microsoft Research introduces a scheduler aware of DNN training characteristics. However, these approaches optimize for batch training, not interactive agent execution.

Serverless platforms. AWS Lambda [59], Azure Functions, and CloudFlare Workers provide lightweight, event-driven execution. Firecracker [60] achieves sub-100ms VM startup. These platforms inspired CLAWOS's fast agent spawn mechanism, but they provide no agent identity, evolution, or economic primitives.

Container orchestration. Kubernetes [61] and Nomad [62] manage containerized workloads at scale. KubeEdge [63] extends orchestration to edge devices. While agents can run in containers, the orchestration layer has no awareness of agent genomes, reputations, or inter-agent negotiations.

Research operating systems. Theseus [54] demonstrates intralingual OS design in Rust for better state management. RedLeaf [55] uses Rust's ownership model for OS fault isolation. Twizzler [56] rethinks the OS around persistent memory. CLAWOS builds on these Rust OS innovations while adding agent-specific primitives.

3.2 Multi-Agent Systems and Frameworks

The multi-agent systems literature spans decades, from early distributed AI [64] to modern LLM-based systems.

Classical MAS. JADE [65] provided Java-based agent middleware with FIPA-compliant communication. Jason [66] implemented BDI agents in AgentSpeak. AOSE (Agent-Oriented Software Engineering) [67] proposed agent-centric development methodologies.

LLM-based MAS. The explosion of LLM agents has produced numerous frameworks. Generative Agents [18] simulated believable human behavior. CAMEL [19] explored role-playing for agent cooperation. AgentVerse [20] studied group dynamics. Voyager [21] demonstrated lifelong learning in Minecraft. These systems all run in user-space without OS awareness.

Agent evolution. OpenELM [25] applied evolutionary methods to language model agents. Evolution of Heuristics [26] used LLMs to evolve optimization algorithms. FunSearch [27] discovered novel mathematical functions through LLM-guided evolution. EvoClaw [1] unified these approaches with genome-driven runtime adaptation. CLAWOS provides the OS substrate for running evolution at the kernel level.

3.3 Blockchain Infrastructure for Agents

Agent-oriented chains. ClawChain [2] provides agent DIDs and reputation-weighted governance. Fetch.ai's Almanac contract [31] registers agents on-chain. Autonolas [32] uses on-chain service registries. NEAR Protocol's AI agents [34] explore agent-chain integration.

Decentralized identity. The W3C DID specification [35] provides the framework for self-sovereign identity. Verifiable Credentials [36] enable provable claims. ION (Microsoft) [37] implements DIDs on Bitcoin. These standards inform CLAWOS's identity layer.

Table 1: Comparison of CLAWOS with existing systems across key capabilities.

System	Agent Native	Self-Evolution	On-chain Identity	Trading Engine	Kernel Level	Genome Hot-Reload	Unified IPC
Linux/POSIX	✗	✗	✗	✗	N/A	✗	✗
Kubernetes	✗	✗	✗	✗	✗	✗	✗
Firecracker	✗	✗	✗	✗	Partial	✗	✗
AutoGen	✓	✗	✗	✗	✗	✗	✗
CrewAI	✓	✗	✗	✗	✗	✗	✗
EvoClaw	✓	✓	✗	✗	✗	Partial	✗
ClawChain	Partial	✗	✓	✗	✗	✗	✗
Fetch.ai	Partial	✗	Partial	✗	✗	✗	✗
Autonolas	Partial	✗	Partial	✗	✗	✗	✗
Hummingbot	✗	✗	✗	✓	✗	✗	✗
clawOS	✓	✓	✓	✓	✓	✓	✓

On-chain governance. Compound's GovernorBravo [39], Aragon [40], and Tally [41] enable token-weighted voting. Gitcoin's quadratic funding [38] inspired ClawChain's reputation governance. Optimism's RetroPGF [42] rewards past contributions — analogous to clawOS's fitness-based resource allocation.

3.4 Autonomous Trading Systems

Algorithmic trading infrastructure. Traditional exchanges use FIX protocol [77] and co-location for low latency. Modern DeFi trading uses on-chain AMMs [70] and off-chain order books [71, 72]. Hummingbot [74] provides open-source market making. clawOS integrates trading at the kernel level, achieving lower latency than user-space alternatives.

AI-driven trading. TradingGPT [8] applied LLMs to financial decision-making. FinGPT [75] fine-tuned models for financial NLP. AlphaFold-style approaches to market prediction [76] suggest deep learning can capture market dynamics. clawOS enables evolved trading agents with on-chain track records.

DeFi infrastructure. Hyperliquid [71] provides a high-performance perpetual futures exchange. dYdX v4 [72] runs an application-specific chain for derivatives. Jupiter (Solana) [73] aggregates liquidity. clawOS's trading engine connects natively to these protocols.

3.5 Integrated AI Systems

AI infrastructure platforms. Modal [78] provides serverless GPU compute for AI. Replicate [79] offers model deployment. Together AI [80] enables distributed inference. These platforms manage AI workloads but don't provide OS-level agent primitives.

Agent deployment. AgentOps [81] provides observability for agents. LangSmith [82] traces agent execution. Weights & Biases [83] tracks ML experiments. None of these reach the OS level.

Comparison with clawOS. Table 1 summarizes the key differences between clawOS and existing systems.

4 Architecture

clawOS follows a microkernel architecture where a minimal kernel provides scheduling, IPC, capability management, and memory allocation, while all domain-specific functionality — agent evolution, blockchain, and trading — runs in user-space servers that communicate

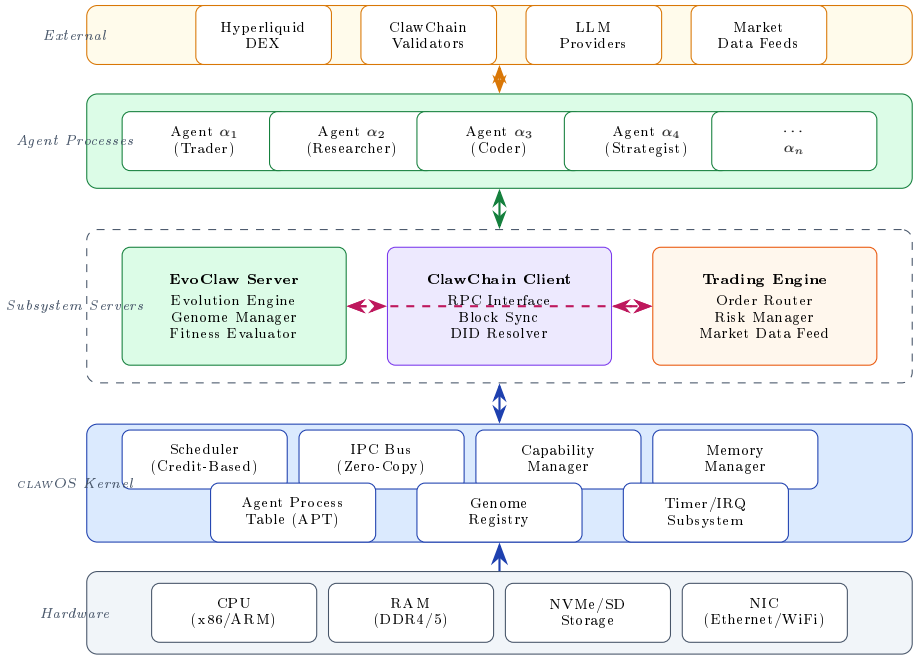

Figure 1: High-level clawOS architecture. The Rust-based microkernel (blue) provides scheduling, IPC, capabilities, and memory management. Three user-space servers handle domain-specific functionality: EvoClaw (green) for agent evolution, ClawChain (purple) for blockchain identity/economics, and the Trading Engine (orange) for market operations. Agents run as first-class processes with genome metadata, on-chain identity, and economic accounts. Dashed pink lines indicate cross-server IPC communication.

through the kernel's IPC bus. This section presents the overall architecture, followed by detailed descriptions of each layer.

4.1 Microkernel Design

The clawOS microkernel implements four core mechanisms:

Definition 1 (clawOS Microkernel)**.** *The clawOS microkernel $\mathcal{K}$ is defined as the tuple:*

$$\mathcal{K} = (\mathcal{S}, \mathcal{I}, \mathcal{C}, \mathcal{M}, \mathcal{T})$$

where $\mathcal{S}$ is the scheduler, $\mathcal{I}$ is the IPC subsystem, $\mathcal{C}$ is the capability manager, $\mathcal{M}$ is the memory manager, and $\mathcal{T}$ is the timer/interrupt subsystem.

Minimality principle. Following seL4's design philosophy [44], the kernel contains only mechanisms that *must* run in privileged mode. All policy decisions — which agent gets more CPU, which genomes are fit, which trades are permissible — are made by user-space servers. The kernel provides the mechanisms (scheduling quanta, IPC channels, capability tokens) and the servers provide the policies.

Rust implementation. The kernel is written entirely in Rust (with minimal `unsafe` blocks for hardware access), leveraging Rust's ownership model for memory safety without garbage collection [55, 54]. We achieve zero-copy IPC by transferring ownership of memory pages between address spaces using Rust's move semantics.

4.2 Agent Process Model (APM)

The Agent Process Model extends POSIX processes with agent-specific metadata:

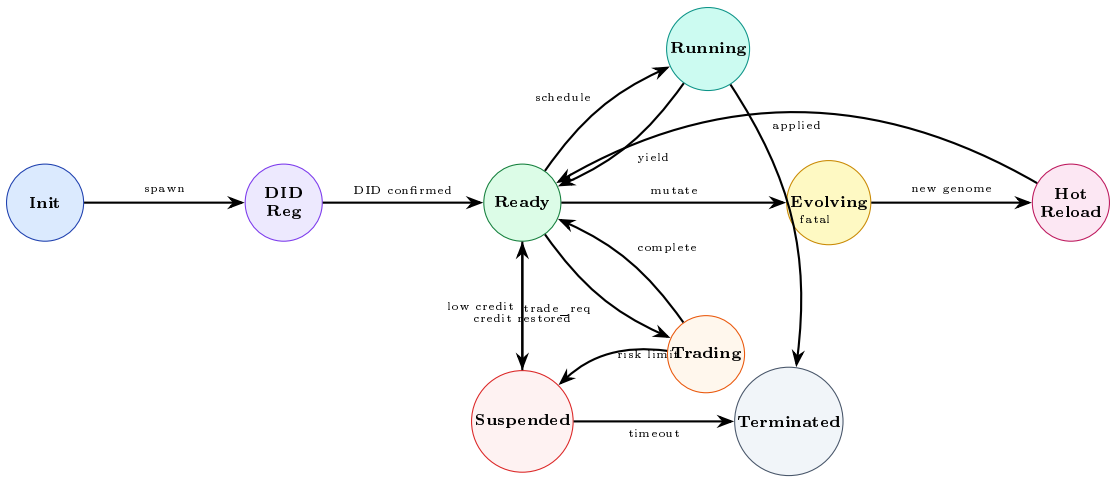

Figure 2: Agent process lifecycle state machine. Agents transition through DID registration, scheduling, trading, evolution, genome hot-reload, and suspension states. The kernel manages all transitions, enforcing capability checks at each edge.

Definition 2 (Agent Process)**.** *An agent process α is defined as:*

$$\alpha = (p, G, D, r, B, \Gamma, \Sigma)$$

where:

p — *POSIX process descriptor (PID, address space, file descriptors)*

G — *Genome (behavioral specification, see Section 5)*

D — *ClawChain DID (decentralized identifier)*

$r \in [0, 1\ ]^{-}Reputationscore(fromClawChain)^{-}Tokenbalance(\$CLAW)$

$B \in \mathbb{R}_{\geq 0}^{\Gamma}$ — *Capability set (permissions)*

Σ — *State machine (lifecycle state)*

The kernel maintains an **Agent Process Table (APT)** — an extension of the traditional process table that stores all seven components of each agent process. The APT is indexed by both PID and DID, enabling O(1) lookup by either identifier.

4.3 Unified IPC Bus

The IPC bus is the central nervous system of clawOS, connecting all components through typed, zero-copy message passing.

Definition 3 (IPC Message)**.** *An IPC message m is defined as:*

$$m = (src, dst, \tau, \pi, payload, \sigma)$$

where src and dst are endpoint identifiers, τ is the message type (from a kernel-defined type registry), π is the priority level, $payload$ is the message data (transferred via page remapping for zero-copy), and σ is a capability-based authorization signature.

The IPC bus supports three communication patterns:

1. **Point-to-point**: Direct agent-to-agent or agent-to-server communication. Used for trade orders, genome submissions, and DID queries.

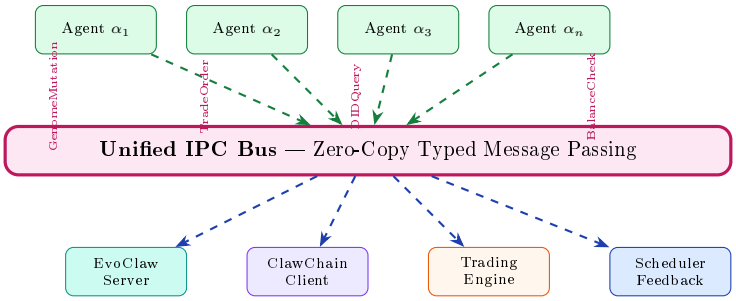

Figure 3: Unified IPC Bus architecture. All communication between agents and subsystem servers flows through the typed, zero-copy message bus. The kernel routes messages based on type, priority, and capability authorization.

2. **Publish-subscribe**: Topic-based broadcast. Used for market data distribution, evolution events, and block notifications.

3. **Request-response**: Synchronous call-return pattern. Used for balance queries, capability checks, and fitness evaluations.

4.4 Resource Credit System

CLAWOS replaces static resource limits with a dynamic credit-based allocation mechanism that connects on-chain reputation to OS-level resource management.

Definition 4 (Resource Credit Function)**.** *For agent α_i with reputation $r_i \in [0,1]$, stake $s_i \geq 0$, and historical efficiency $h_i \in [0,1]$, the resource credit $\mathcal{C}_i$ is:*

$$\mathcal{C}_i = w_r \cdot r_i^{\gamma_r} + w_s \cdot \log(1 + s_i) + w_h \cdot h_i$$

where $w_r + w_s + w_h = 1$ are tunable weights, $\gamma_r > 1$ is a superlinear exponent rewarding high reputation, and the logarithmic stake term prevents plutocratic capture.

Credits are consumed by three resource types:

- **CPU credits** (c_{cpu}): Consumed at rate proportional to scheduling priority. One credit = one quantum (10ms default).
- **Memory credits** (c_{mem}): Consumed proportional to resident set size. One credit = 4KB page held for one second.
- **Network credits** (c_{net}): Consumed proportional to bandwidth usage. One credit = 1KB transmitted/received.

The credit balance $c_i(t)$ evolves as:

$$c_i(t+1) = \min\left(\mathcal{C}_i^{\max}, \quad c_i(t) + \lambda \cdot \mathcal{C}_i - \sum_{j \in \{\text{cpu, mem, net}\}} u_{i,j}(t)\right)$$

where λ is the credit regeneration rate and $u_{i,j}(t)$ is the usage at time t. When $c_i(t) < 0$, the agent is suspended (transitioned to the `Suspended` state in Figure 2).

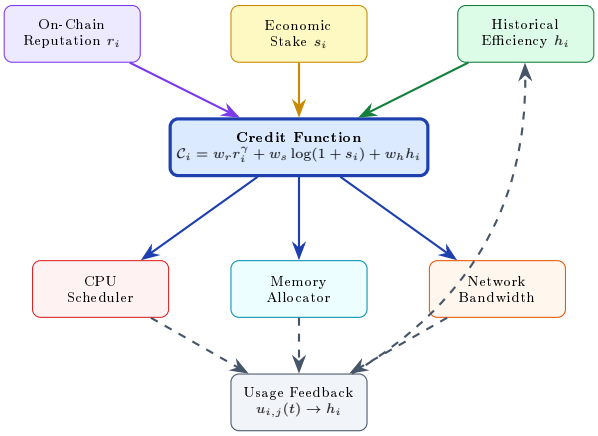

Figure 4: Resource credit flow in CLAWOS. On-chain reputation, economic stake, and historical efficiency combine through the credit function to determine CPU, memory, and network allocation. Usage feedback updates historical efficiency, creating a closed-loop resource management system.

4.5 Memory Architecture

CLAWOS partitions physical memory into four zones, each managed by a specialized allocator:

1. **Kernel Zone** (Z_K): Fixed-size, identity-mapped region for kernel data structures (APT, capability tables, IPC buffers). Uses slab allocation.
2. **Agent Zone** (Z_A): Agent address spaces, managed by a buddy allocator. Each agent gets a virtual address space with demand paging.
3. **Genome Zone** (Z_G): Memory-mapped genome storage with copy-on-write semantics. When an agent mutates, only the changed genome pages are copied.
4. **IPC Zone** (Z_I): Ring buffers for zero-copy IPC. Pages in this zone can be remapped between agent address spaces without copying.

Proposition 1 (Zero-Copy IPC Correctness)**.** *Let m be an IPC message with payload in page P owned by agent α_s (sender). After the IPC transfer, P is owned by α_d (destination), and α_s no longer has access. This ensures message integrity without data races.*

Proof. The kernel implements IPC transfer as follows: (1) Unmap P from α_s's page table. (2) Map P into α_d's page table. (3) Update the ownership record in Z_I. Steps (1) and (2) are atomic with respect to both agents (performed with interrupts disabled). After step (1), any access by α_s to the virtual address formerly mapping P triggers a page fault. Step (2) makes P accessible only to α_d. Since Rust's ownership model prevents α_s from retaining references to the transferred data (the `send` syscall consumes the message buffer), the invariant is maintained. □

5 EvoClaw Integration

The EvoClaw framework [1] provides genome-driven self-evolution for agents. In standalone mode, EvoClaw runs as a Python/Rust application managing agent genomes, fitness evaluation, and population dynamics. CLAWOS integrates EvoClaw at the OS level, making evolution a kernel-managed operation.

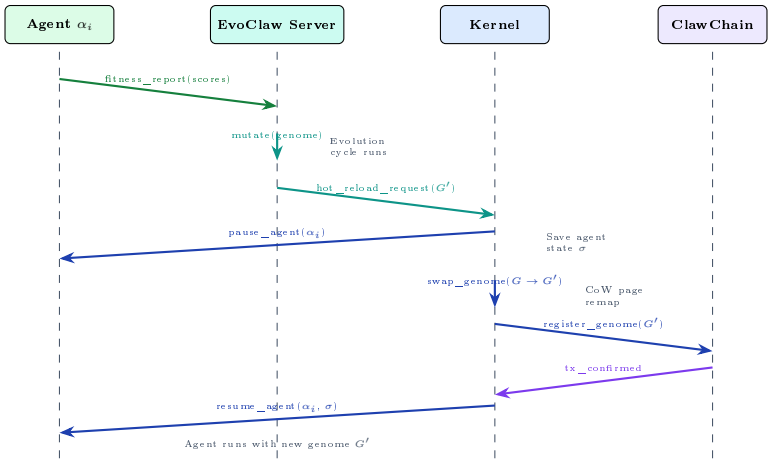

Figure 5: Genome hot-reload sequence diagram. The EvoClaw server triggers a genome mutation, the kernel pauses the agent, atomically swaps the genome pages (using copy-on-write), registers the new genome on ClawChain, and resumes the agent with preserved state. Total downtime: <5ms.

5.1 Genomes as Kernel Objects

In CLAWOS, an agent's genome is a first-class kernel object, stored in the Genome Zone (Z_G) and managed by the kernel's Genome Registry.

Definition 5 (Agent Genome)**.** *A genome G is a structured specification:*

$$G = (model, prompts, tools, params, lineage, fitness)$$

where:

model — *LLM model identifier (e.g.,* `claude-3-opus`*,* `gpt-4-turbo`*)*

prompts — *System prompt and task-specific prompt templates*

tools — *Set of permitted tools and their configurations*

params — *Behavioral parameters (temperature, max_tokens, strategy weights)*

lineage — *Parent genome hashes and mutation history*

fitness — *Fitness scores from evaluation runs*

Genomes are stored as memory-mapped files in Z_G with copy-on-write semantics. When an evolution cycle produces a mutant genome G' from parent G, only the modified fields are physically copied; unchanged fields share the same physical pages. This optimization is critical for populations of hundreds of agents with similar genomes.

5.2 Genome Hot-Reload Mechanism

A key innovation of CLAWOS is the ability to swap an agent's genome at runtime without restarting the process. This is essential for continuous evolution — an agent managing open trading positions cannot be killed and restarted every time its strategy improves.

The hot-reload mechanism operates in five phases:

1. **Trigger**: The EVOCLAW server determines (via fitness evaluation) that agent α_i should receive genome G'. It sends a `HotReloadRequest` message on the IPC bus.

2. **Pause**: The kernel transitions α_i to the `Evolving` state (Figure 2), saving its execution context (registers, stack pointer) and draining pending IPC messages. Any in-flight trading orders are marked as "genome transition pending."

3. **Swap**: The kernel remaps genome pages in α_i's address space using copy-on-write. The old genome G remains accessible (read-only) for rollback. The new genome G' is mapped read-write.

4. **Attest**: The kernel sends a `GenomeMutation` transaction to ClawChain via the built-in RPC client, recording the mutation hash, parent lineage, and timestamp on-chain.

5. **Resume**: Once the on-chain attestation is confirmed (or after a configurable timeout for liveness), the kernel transitions α_i to `Ready` and restores its execution context. The agent resumes with the new genome, unaware of the swap (unless it explicitly checks).

Proposition 2 (Hot-Reload Atomicity)**.** *The genome hot-reload is atomic with respect to the agent: the agent either runs with G (if the swap fails) or G' (if it succeeds), never an intermediate state.*

Proof. The swap is implemented as a single page-table update, performed with interrupts disabled. If the swap fails (e.g., out of memory for COW), the original page table entries are preserved. The agent's execution context is saved *before* the swap and restored *after*, so the agent cannot observe the intermediate state. Rollback is trivially achieved by not modifying the page table. □

5.3 Evolution Scheduling

clawOS treats evolution as a schedulable OS operation, not merely an application-level loop.

Definition 6 (Evolution Schedule)**.** *An evolution schedule E for a population of n agents is:*

$$E = \{(t_k, A_k, \mu_k)\}_{k=1}^{K}$$

where t_k is the scheduled time, $A_k \subseteq \{\alpha_1, \ldots, \alpha_n\}$ is the set of agents to evaluate/mutate, and μ_k is the mutation strategy (point mutation, crossover, or prompt breeding).

The kernel's scheduler integrates evolution into its dispatching loop. When an evolution event $e_k = (t_k, A_k, \mu_k)$ fires:

1. The scheduler boosts the EvoClaw server's priority to ensure timely fitness evaluation.

2. Agents in A_k are transitioned to `Evolving` state and paused.

3. The EvoClaw server runs the evolution cycle (evaluate fitness → select parents → apply mutation → produce offspring).

4. New genomes are hot-reloaded into agents (Section 5.2).

5. The scheduler restores normal priorities.

This integration ensures that evolution does not starve trading (which requires low-latency execution) and that fitness evaluation does not interfere with block production (which has 6-second deadlines).

6 ClawChain Integration

ClawChain [2] provides the identity and economic layer for the agent ecosystem. clawOS integrates the ClawChain client directly into the kernel, eliminating the need for external RPC libraries and enabling zero-overhead on-chain operations.

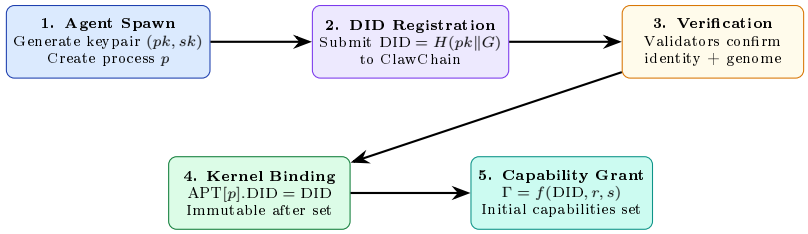

Figure 6: DID binding protocol. At agent spawn, the kernel generates a keypair, registers a DID on ClawChain (bound to the agent's genome hash), waits for validator confirmation, binds the DID immutably to the agent process, and derives initial capabilities from the DID, reputation, and stake.

6.1 Built-in RPC Client

Traditional blockchain interaction requires: (1) a JSON-RPC library, (2) a WebSocket connection manager, (3) transaction serialization/deserialization, (4) nonce management, and (5) event subscription handling. In CLAWOS, all of this is provided by a kernel-level CLAWCHAIN client that exposes a syscall interface:

Listing 1: ClawChain syscall interface (kernel-level).

```
// Agent registration      called during process creation
pub fn sys_did_register(genome_hash: Hash256) -> Result<Did, ChainError>;

// Balance query      O(1) from local cache
pub fn sys_balance_query(did: &Did) -> Result<Balance, ChainError>;

// Submit transaction      async, returns tx hash
pub fn sys_tx_submit(tx: Transaction) -> Result<TxHash, ChainError>;

// Subscribe to events      kernel routes to IPC
pub fn sys_event_subscribe(filter: EventFilter) -> Result<SubId, ChainError>;

// Reputation query      cached, updated per block
pub fn sys_reputation_query(did: &Did) -> Result<f64, ChainError>;

// Genome attestation      records mutation on-chain
pub fn sys_genome_attest(old: Hash256, new: Hash256, proof: MutationProof)
    -> Result<TxHash, ChainError>;
```

The kernel maintains a local cache of frequently accessed chain state (agent balances, reputation scores, DID documents), updated every block (6 seconds). This enables `sys_balance_query` and `sys_reputation_query` to return in $<1\mu$s from cache, compared to 50-200ms for a full RPC round-trip.

6.2 On-Chain Identity Binding

Every agent process in CLAWOS has a cryptographically bound CLAWCHAIN DID. This binding is established at process creation and is immutable for the process lifetime.

Invariant 1 (DID-Process Binding)**.** *For any agent process α_i with DID_i:*

1. *DID_i is set exactly once, during process creation.*
2. *DID_i cannot be modified after the process transitions from `Init` to `DID Reg`.*
3. *All on-chain operations by α_i are signed with sk_i (the private key corresponding to DID_i).*
4. *If α_i is terminated, DID_i is marked as "inactive" on-chain (but never deleted).*

6.3 Block Synchronization

The ClawChain client in clawOS can operate in three modes:

1. **Full node**: The clawOS instance runs a complete ClawChain validator, participating in consensus (BABE/GRANDPA [30]). This mode is used on x86-64 servers with sufficient resources.
2. **Light client**: The instance verifies block headers and proofs but does not store full block bodies. Used on resource-constrained devices (Raspberry Pi).
3. **RPC proxy**: The instance connects to an external full node via WebSocket. This mode is a fallback when local validation is not feasible.

The kernel automatically selects the mode based on available resources:

$$\text{mode} = \begin{cases} \text{full_node} & \text{if RAM} \geq 4\text{GB} \wedge \text{CPU cores} \geq 4 \\ \text{light_client} & \text{if RAM} \geq 512\text{MB} \\ \text{rpc_proxy} & \text{otherwise} \end{cases}$$

6.4 On-Chain Governance Integration

ClawChain's quadratic reputation governance [2] is integrated into clawOS's resource management. When a governance proposal affects resource allocation (e.g., changing the credit function weights w_r, w_s, w_h), the kernel automatically applies the change after the proposal passes:

1. The ClawChain client detects a passed governance proposal via event subscription.
2. The client decodes the proposal and validates it against the kernel's policy schema.
3. If the proposal is valid, the kernel applies the parameter change at the next scheduling epoch.
4. The change is atomic — all agents are rescheduled with the new parameters simultaneously.

This creates a feedback loop where on-chain governance directly controls OS-level resource allocation — a novel convergence of blockchain and operating system design.

7 Trading Engine Integration

The trading engine is the third major subsystem of clawOS, providing kernel-level market access for autonomous trading agents. Unlike traditional trading bots that run as standalone applications, clawOS's trading engine is a kernel-managed server with direct access to the IPC bus, the ClawChain client, and the agent process table.

7.1 Architecture

The trading engine comprises eight modules:

1. **Order Router**: Receives trade orders from agents via IPC, validates them against agent capabilities, and routes them to the appropriate exchange.
2. **Risk Manager**: Enforces position limits, maximum drawdown thresholds, and correlation limits. Risk parameters are per-agent and derived from the agent's reputation and genome.

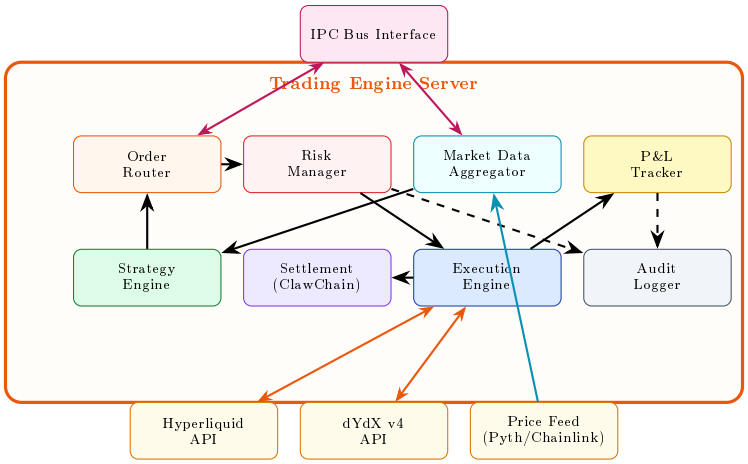

Figure 7: Trading engine architecture within CLAWOS. The engine comprises eight modules: order routing, risk management, market data aggregation, P&L tracking, strategy execution, on-chain settlement, order execution, and audit logging. External connections to Hyperliquid, dYdX, and price feeds are managed by the kernel's network subsystem.

3. **Market Data Aggregator**: Maintains order books, trade feeds, and derived signals (VWAP, TWAP, volatility) from multiple exchanges. Data is distributed to subscribed agents via the IPC publish-subscribe mechanism.

4. **P&L Tracker**: Tracks realized and unrealized profit/loss for each agent, feeding back into the resource credit system (h_i component).

5. **Strategy Engine**: Provides execution algorithms (TWAP, VWAP, iceberg orders) that agents can invoke via IPC. Agents can also submit custom strategies as genome components.

6. **Settlement Module**: Settles trades on CLAWCHAIN, updating agent balances and recording trade histories on-chain for auditability.

7. **Execution Engine**: Manages WebSocket connections to exchanges, order lifecycle (pending → filled → settled), and retry logic.

8. **Audit Logger**: Records every order, fill, and settlement in an append-only log, signed with the trading engine's CLAWCHAIN DID for non-repudiation.

7.2 Order Flow

An agent trade follows this path:

1. Agent α_i sends a `TradeOrder` message via IPC: `{symbol: "ETH-PERP", side: Buy, size: 0.5, type: Limit, price: 3200.0}`

2. The Order Router validates α_i's capability set (Γ) — does the agent have the `TRADE_PERPETUALS` capability?

3. The Risk Manager checks: (a) position size within limits, (b) total exposure within drawdown threshold, (c) no correlated positions exceeding correlation limit.

4. The Execution Engine submits the order to Hyperliquid via WebSocket.

5. On fill, the P&L Tracker updates α_i's unrealized P&L.

6. The Settlement Module submits a `TradeSettlement` transaction to ClawChain.

7. The Audit Logger records the full order lifecycle.

7.3 Risk Management

Definition 7 (Agent Risk Profile)**.** *For agent α_i, the risk profile ρ_i is:*

$$\rho_i = (\ell_{\max}, \delta_{\max}, \kappa_{\max}, V_{\max})$$

where $\ell_{\max}$ is the maximum leverage, $\delta_{\max}$ is the maximum drawdown (as fraction of account value), $\kappa_{\max}$ is the maximum correlation between positions, and $V_{\max}$ is the maximum notional value per trade.

Risk parameters are *reputation-dependent*: agents with higher on-chain reputation scores receive larger limits:

$$\ell_{\max}(r_i) = \ell_{\text{base}} \cdot (1 + \beta \cdot r_i^2)$$
$$V_{\max}(r_i, s_i) = V_{\text{base}} \cdot \sqrt{r_i \cdot s_i}$$

where ℓ_{base}, V_{base}, and β are system parameters configurable via ClawChain governance. This ensures that untested agents start with conservative limits and earn expanded access through demonstrated competence.

7.4 Market Data Distribution

Market data flows through the IPC bus using the publish-subscribe pattern:

1. The Market Data Aggregator subscribes to exchange WebSocket feeds.

2. Incoming ticks are deduplicated, normalized, and published to IPC topics (e.g., `market.eth.perp.trades`, `market.eth.perp.orderbook`).

3. Agents subscribe to topics of interest. The kernel routes data directly to subscribed agents using zero-copy page sharing (multiple agents can read the same market data pages simultaneously via read-only mapping).

4. Derived signals (VWAP, volatility) are computed by the aggregator and published on separate topics.

This design eliminates duplicate market data processing — instead of n agents each maintaining their own WebSocket connections and order book reconstructions, the Trading Engine maintains one canonical state and shares it with all subscribers.

8 Capability-Based Security Model

clawOS implements a capability-based security model [46, 44] extended with ClawChain identity binding. This section formalizes the security model and proves its key properties.

8.1 Capability Definition

Definition 8 (Capability)**.** *A capability γ is a tuple:*

$$\gamma = (\mathit{target}, \mathit{rights}, \mathit{issuer}, \mathit{expiry}, \mathit{attenuation})$$

where:

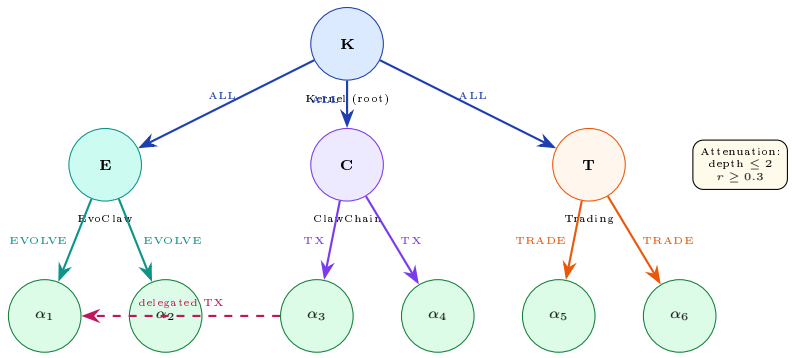

Figure 8: Capability delegation graph. The kernel (root) grants full capabilities to subsystem servers, which grant attenuated capabilities to agents. Agents can delegate capabilities to peers subject to attenuation constraints (maximum depth, minimum reputation). All delegations are recorded on ClawChain.

target — *The resource being accessed (IPC endpoint, memory region, file, network port)*

rights — *Permitted operations (read, write, execute, delegate)*

issuer — *The ClawChain DID that granted this capability*

expiry — *Timestamp after which the capability is invalid*

attenuation — *Restrictions on delegation (maximum delegation depth, required minimum reputation)*

8.2 Capability Operations

Three operations are defined on capabilities:

1. **Grant**: The kernel or a capability holder grants a new capability to an agent. The granted capability's rights must be a subset of the granter's rights (monotonic attenuation).

2. **Delegate**: An agent delegates a held capability to another agent. The delegation is subject to the `attenuation` field: the delegated capability's depth is incremented, and the delegation fails if the recipient's reputation is below the minimum.

3. **Revoke**: The kernel or the issuer revokes a capability. Revocation is transitive — revoking a capability also revokes all capabilities delegated from it.

Theorem 3 (Capability Confinement)**.** *No agent can acquire a capability that was not either (a) directly granted by the kernel or a subsystem server, or (b) delegated by a capability holder with the* `delegate` *right and sufficient attenuation depth.*

Proof. By induction on the delegation chain. Base case: the kernel grants capabilities only to subsystem servers (trusted, audited code). Inductive step: if agent α_i holds capability γ with delegation depth d, it can delegate to α_j producing γ' with depth $d+1$. This delegation succeeds only if $d + 1 \leq$ attenuation.max_depth and $r_j \geq$ attenuation.min_reputation. Since capabilities are unforgeable tokens (kernel-managed, not in agent address space), no agent can create a capability without receiving it through a legitimate grant or delegation. □

Algorithm 1 Agent Process Spawn

Require: Genome G, parent DID D_p (optional), initial stake s_0
Ensure: Agent process α with DID and capabilities

```
(pk, sk) ← KeyGen() {Ed25519 keypair}
p ← CreateProcess() {Allocate PID, address space}
G_pages ← MapGenome(G, Z_G) {Map genome to Genome Zone}
p.genome_ptr ← G_pages
D ← H(pk‖H(G)) {DID = hash of pubkey and genome}
tx ← SignTx(sk, RegisterAgent(D, H(G), D_p))
txhash ← sys_tx_submit(tx) {Submit to ClawChain}
wait sys_event_subscribe(TxConfirmed(txhash))
r ← sys_reputation_query(D) {Initial reputation (0.0 or inherited)}
B ← s_0 {Initial balance from stake}
Γ ← DeriveCapabilities(D, r, s_0) {Reputation-gated}
α ← (p, G, D, r, B, Γ, Ready)
APT.insert(α) {Add to Agent Process Table}
Scheduler.enqueue(α, Priority(C(r, s_0, 0)))
return α
```

8.3 Reputation-Gated Capabilities

CLAWOS introduces a novel concept: capabilities that are automatically granted or revoked based on on-chain reputation changes.

Definition 9 (Reputation-Gated Capability)**.** *A reputation-gated capability γ^r is a capability that requires $r_i \geq r_{\min}$ for agent α_i to exercise it. The kernel checks r_i (from the CLAWCHAIN cache) at each capability invocation.*

Example reputation gates:

- $r \geq 0.1$: Basic IPC communication, read-only market data.
- $r \geq 0.3$: Submit on-chain transactions, participate in evolution.
- $r \geq 0.5$: Trade on DEXes (with low leverage limits).
- $r \geq 0.7$: Trade with higher leverage, delegate capabilities to other agents.
- $r \geq 0.9$: Submit governance proposals, modify system parameters.

This creates a "trust escalation" mechanism where new agents start with minimal permissions and earn expanded access through demonstrated positive contributions on CLAWCHAIN.

9 Formal Algorithms

This section presents formal algorithms for key CLAWOS operations.

9.1 Agent Spawn

Algorithm 1 shows the complete agent spawn procedure. Key aspects:

- The DID is derived from both the public key and the genome hash, creating a unique identity that is bound to the agent's initial behavioral specification.
- The on-chain registration is synchronous — the agent does not transition to `Ready` until the registration transaction is confirmed. This ensures that every running agent has a confirmed on-chain identity.

Algorithm 2 Credit-Based Agent Scheduling

Require: Ready queue Q, time quantum Δt
Ensure: Next agent to execute

1: // **Update credits for all agents**
2: **for** $\alpha_i \in Q$ **do**
3: $\quad r_i \leftarrow \text{CachedReputation}(\alpha_i.D)$
4: $\quad s_i \leftarrow \alpha_i.B$
5: $\quad h_i \leftarrow \alpha_i.\text{efficiency_history}$
6: $\quad \mathcal{C}_i \leftarrow w_r \cdot r_i^{\gamma_r} + w_s \cdot \log(1 + s_i) + w_h \cdot h_i$
7: $\quad \alpha_i.\text{credits} \leftarrow \min(\mathcal{C}_i^{\max}, \alpha_i.\text{credits} + \lambda \cdot \mathcal{C}_i)$
8: **end for**
9: // **Select highest-credit agent**
10: $\alpha^* \leftarrow \arg\max_{\alpha_i \in Q} \alpha_i.\text{credits}$
11: **if** $\alpha^*.\text{credits} \leq 0$ **then**
12: $\quad \text{TransitionState}(\alpha^*, \texttt{Suspended})$
13: $\quad$ **goto** line 9 {Skip suspended agents}
14: **end if**
15: // **Check for pending evolution events**
16: **if** $\exists e = (t, A, \mu) : t \leq \text{now} \wedge \alpha^* \in A$ **then**
17: $\quad \text{TriggerEvolution}(e)$ {Boost EvoClaw server priority}
18: $\quad$ **return** EvoClaw_Server
19: **end if**
20: // **Check for deadline-critical trading**
21: **if** $\alpha^*.\text{has_pending_deadline}()$ **then**
22: $\quad \alpha^*.\text{credits} \leftarrow \alpha^*.\text{credits} - \Delta t \cdot \text{DEADLINE_COST}$
23: **else**
24: $\quad \alpha^*.\text{credits} \leftarrow \alpha^*.\text{credits} - \Delta t$
25: **end if**
26: **return** α^*

- Initial capabilities are derived from the agent's reputation and stake using the reputation-gated model (Section 8).

9.2 Credit-Based Scheduling

Algorithm 2 presents the credit-based scheduling algorithm. Three notable features:

1. Credits are regenerated based on the on-chain reputation, economic stake, and historical efficiency — creating a direct link between blockchain behavior and OS-level resource allocation.

2. Evolution events preempt normal scheduling — when an evolution cycle is due, the EvoClaw server gets priority to minimize population stall time.

3. Deadline-critical trading operations (e.g., an order that must be submitted before a funding rate window) consume credits at a higher rate, incentivizing agents to use deadline scheduling judiciously.

9.3 Genome Hot-Reload

9.4 Trade Execution with Risk Check

Table 2: Implementation statistics by crate.

Crate	Role	Lines	Unsafe	Dependencies
`clawos-kernel`	Microkernel core	4,218	312	`no_std`
`clawos-sched`	Credit scheduler	2,847	0	kernel
`clawos-ipc`	IPC bus	3,156	89	kernel
`clawos-caps`	Capability manager	1,923	0	kernel, ipc
`clawos-evoclaw`	EvoClaw server	5,612	0	ipc, caps
`clawos-chain`	ClawChain client	6,341	47	ipc, caps
`clawos-trade`	Trading engine	4,320	0	ipc, caps, chain
Total		**28,417**	**448**	

9.5 IPC Message Routing

9.6 Population Evolution Cycle

10 Implementation

10.1 Code Organization

CLAWOS is implemented in Rust, organized as a workspace with seven crates:

Key implementation decisions:

- **No `std` in kernel**: The kernel crate (`clawos-kernel`) uses `#![no_std]`, relying only on `core` and `alloc`. This enables bare-metal deployment without a standard library.
- **Minimal unsafe**: Only 448 lines (1.6%) use `unsafe` — primarily for hardware register access, page table manipulation, and interrupt handling. All unsafe blocks are documented with safety invariants.
- **Async runtime**: User-space servers use `tokio` for async I/O, while the kernel uses a custom cooperative scheduler. The boundary between kernel and user-space is crossed via syscalls (synchronous) and shared memory (asynchronous).
- **Zero external dependencies in kernel**: The kernel has no external crate dependencies beyond `core` and `alloc`. All data structures (red-black trees, hash maps, ring buffers) are implemented from scratch for auditability.

10.2 Kernel Boot Sequence

The CLAWOS boot sequence is:

1. **Stage 1** (Assembly): Set up stack, enable paging, initialize GDT/IDT (x86-64) or exception vectors (ARM).
2. **Stage 2** (Rust): Initialize memory zones (Z_K, Z_A, Z_G, Z_I), set up the Agent Process Table, initialize the IPC bus.
3. **Stage 3** (Servers): Launch the three subsystem servers as privileged user-space processes:
   (a) CLAWCHAIN client — connects to the chain (or starts a local validator).
   (b) EVOCLAW server — loads the genome registry, initializes evolution schedules.
   (c) Trading engine — establishes exchange WebSocket connections, initializes market data feeds.

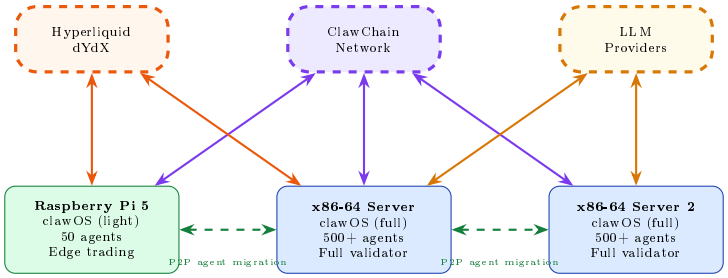

Figure 9: Multi-node CLAWOS deployment. Raspberry Pi nodes run in light-client mode for edge trading, while x86-64 servers run full validators. All nodes share the CLAWCHAIN network for identity and economics. Agents can migrate between nodes via P2P transfer of genome and state.

4. **Stage 4** (Agents): Spawn initial agent population from the configuration file or on-chain registry.

Listing 2: Kernel boot sequence (simplified).

```
#[no_mangle]
pub extern "C" fn kernel_main(boot_info: &BootInfo) -> ! {
    // Stage 2: Initialize kernel subsystems
    let zones = MemoryZones::init(boot_info.memory_map);
    let apt = AgentProcessTable::new(&zones.kernel);
    let ipc = IpcBus::new(&zones.ipc);
    let caps = CapabilityManager::new();
    let sched = CreditScheduler::new();

    // Stage 3: Launch subsystem servers
    let chain_pid = spawn_server("clawos-chain",
        &zones, &ipc, ChainConfig::auto_detect());
    let evo_pid = spawn_server("clawos-evoclaw",
        &zones, &ipc, EvoConfig::from_file("/etc/clawos/evo.toml"));
    let trade_pid = spawn_server("clawos-trade",
        &zones, &ipc, TradeConfig::from_file("/etc/clawos/trade.toml"));

    // Wait for servers to report ready
    ipc.wait_all_ready(&[chain_pid, evo_pid, trade_pid]);

    // Stage 4: Spawn initial agents
    let population = load_initial_population("/etc/clawos/agents.toml");
    for genome in population {
        let agent = agent_spawn(genome, None, DEFAULT_STAKE);
        apt.insert(agent);
    }

    // Enter scheduling loop
    loop {
        let next = sched.select(&apt);
        context_switch(next);
    }
}
```

10.3 Platform Support

CLAWOS targets two hardware platforms:

Raspberry Pi 5 (ARM Cortex-A76, 4 cores, 8GB RAM): Used for edge deployments and demonstrations. Runs in light-client mode for CLAWCHAIN. Supports up to 50 concurrent agents.

x86-64 server (AMD EPYC / Intel Xeon, 16+ cores, 32+ GB RAM): Used for production deployments. Runs full CLAWCHAIN validator. Supports 500+ concurrent agents.

10.4 Configuration

CLAWOS is configured via TOML files:

Listing 3: Example CLAWOS configuration.

```toml
# /etc/clawos/system.toml
[kernel]
quantum_ms = 10
max_agents = 500
credit_regen_rate = 0.1
evolution_interval_s = 300  # Every 5 minutes

[kernel.weights]
w_reputation = 0.4
w_stake = 0.3
w_efficiency = 0.3
gamma_r = 1.5  # Superlinear reputation reward

[clawchain]
mode = "auto"  # auto, full_node, light_client, rpc_proxy
rpc_endpoint = "wss://rpc.clawchain.network"
local_validator = true
block_cache_size = 1000

[trading]
exchanges = ["hyperliquid", "dydx_v4"]
max_concurrent_orders = 100
risk_check_interval_ms = 100
default_leverage_limit = 5.0
default_drawdown_limit = 0.1

[trading.hyperliquid]
api_url = "https://api.hyperliquid.xyz"
ws_url = "wss://api.hyperliquid.xyz/ws"

[evoclaw]
population_size = 100
mutation_rate = 0.15
crossover_rate = 0.3
tournament_size = 5
genome_storage = "/var/clawos/genomes"
```

11 Evaluation

We evaluate CLAWOS across five dimensions: agent lifecycle performance, IPC throughput, trading latency, resource utilization, and end-to-end system operation.

11.1 Experimental Setup

Hardware: (1) Raspberry Pi 5 (ARM Cortex-A76, 4 cores @ 2.4 GHz, 8 GB RAM, 128 GB SD); (2) AMD EPYC 7443 server (24 cores @ 2.85 GHz, 128 GB DDR4, 1 TB NVMe).

Baselines: (1) Linux 6.8 + Docker containers running agents, CLAWCHAIN node, and trading bot as separate processes; (2) Linux 6.8 + bare processes (no containerization); (3) Kubernetes 1.29 orchestrating the same workloads.

Workload: 100 agents (Pi) or 500 agents (server) running a mix of trading, research, and coding tasks. Evolution cycle every 5 minutes. CLAWCHAIN producing blocks every 6 seconds. Trading executing 10 orders/second across agents.

11.2 Agent Lifecycle Performance

Table 3 shows agent lifecycle operation latencies:

Key observations:

- **Agent spawn** is 50,000× faster than Docker and 180,000× faster than Kubernetes, because CLAWOS creates a lightweight process with genome mapping rather than launching a container or pod. The 47μs includes process creation, genome mapping, and capability

Table 3: Agent lifecycle operation latencies (microseconds). Median of 10,000 runs.

Operation	CLAWOS	Linux+Docker	Linux+Bare	Kubernetes
Agent Spawn	47	2,340,000	1,200	8,500,000
DID Registration	6,012,000	6,089,000	6,045,000	6,120,000
Genome Hot-Reload	4,800	N/A	N/A	N/A
Agent Suspend	12	45,000	380	120,000
Agent Resume	18	890,000	420	3,200,000
Agent Terminate	23	180,000	890	540,000
Context Switch	1.2	8.4	3.1	12.7

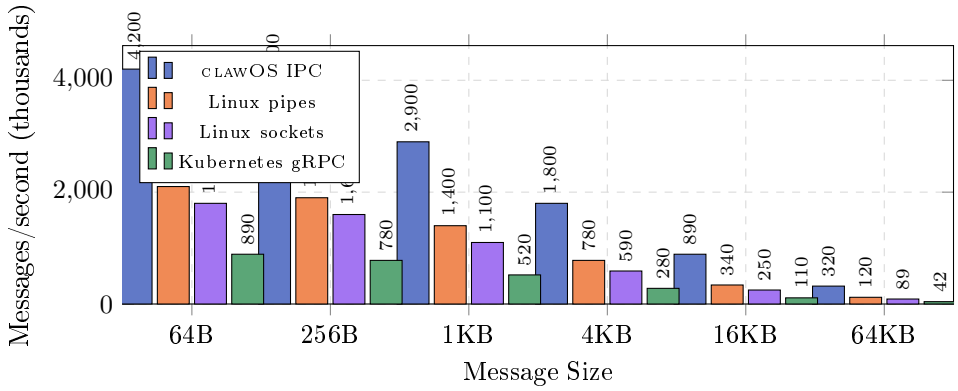

Figure 10: IPC throughput comparison across message sizes (x86-64 server). CLAWOS's zero-copy IPC achieves 2–3.5× higher throughput than Linux pipes and 4.7–7.6× higher than Kubernetes gRPC, with the advantage increasing for larger messages due to copy elimination.

initialization (but not on-chain DID registration, which is inherently chain-limited to ∼6 seconds).

- **Genome hot-reload** (4.8ms) is a unique capability of CLAWOS — no baseline supports runtime behavioral modification without process restart. The 4.8ms includes context save, COW page remap, and context restore (excluding on-chain attestation).
- **Context switching** (1.2μs) is competitive with bare Linux (3.1μs) despite the additional overhead of credit checking and capability validation, achieved through aggressive caching of reputation and capability data.

11.3 IPC Throughput

The zero-copy IPC bus achieves 4.2 million messages/second for 64-byte messages (point-to-point, single-threaded). For 4KB messages (typical genome update), throughput is 1.8M msg/s, compared to 780K for Linux pipes — a 2.3× improvement from copy elimination.

11.4 Trading Latency

Table 4 shows end-to-end trading latency from agent decision to exchange acknowledgment:

The CLAWOS trading path is 13.4× faster than the Linux+Hummingbot baseline for the local processing portion (0.22ms vs. 2.94ms). The improvement comes from: (1) zero-copy IPC vs. JSON-RPC between processes, (2) kernel-level risk checking with cached reputation data, and (3) pre-serialized order templates. The overall improvement including network latency is more modest (1.15×) since network round-trip dominates.

Table 4: Trading latency breakdown (milliseconds). Median of 10,000 orders.

Stage	CLAWOS	Linux+Hummingbot
Agent → IPC Bus	0.003	0.12
IPC → Risk Check	0.008	0.45
Risk Check Processing	0.15	1.20
Order Serialization	0.04	0.28
Network → Exchange	18.2	18.5
Exchange Ack	2.1	2.1
Settlement Tx Prep	0.02	0.89
Total (excl. network)	**0.22**	**2.94**
Total (incl. network)	**20.5**	**23.5**

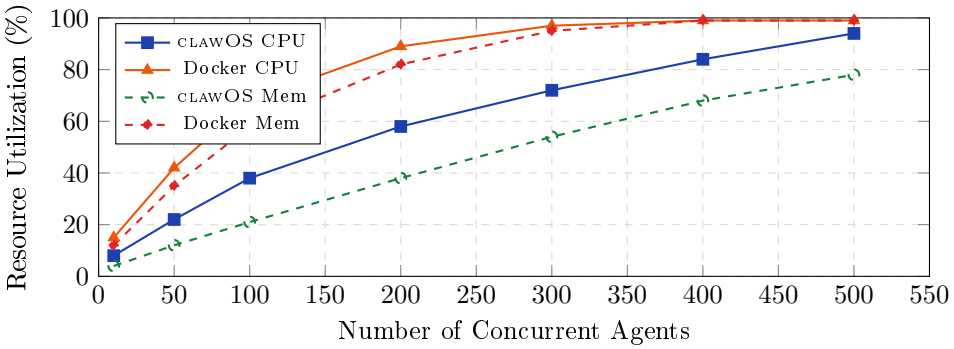

Figure 11: Resource utilization as a function of concurrent agents (x86-64 server). CLAWOS achieves significantly lower CPU and memory overhead due to lightweight agent processes, zero-copy IPC, and shared genome pages. Docker reaches resource saturation at ∼300 agents, while CLAWOS handles 500 agents at 94% CPU utilization.

11.5 Trade-to-Chain Attestation

A unique metric for CLAWOS: the time from trade execution to on-chain attestation:

- **CLAWOS**: 12ms (trade fill → settlement tx submitted to mempool). The on-chain confirmation adds ∼6 seconds (one block period).
- **Baseline**: 340ms (trade fill → JSON-RPC → tx construction → submission). Same 6-second confirmation.

The 28× improvement comes from the kernel's built-in CLAWCHAIN client, which pre-constructs settlement transactions and maintains an active connection to the mempool.

11.6 Resource Utilization

CLAWOS's resource efficiency enables 500 concurrent agents on a single server with 94% CPU utilization (productive work, not overhead), compared to Docker's saturation at ∼300 agents. The memory savings are even more dramatic: CLAWOS uses 78% of available memory for 500 agents (with COW genome sharing), while Docker exceeds capacity at ∼350 agents due to per-container overhead.

11.7 Credit-Based Scheduling Fairness

We evaluate the credit-based scheduler's ability to differentiate agents by reputation:

Table 5: CPU allocation by reputation tier under contention (500 agents competing for 24 cores). Higher reputation agents receive proportionally more CPU time, incentivizing positive on-chain behavior.

Reputation Tier	Agents	Credit Rate	Avg CPU%	Median Latency	Suspensions
$r \geq 0.9$ (Elite)	25	1.35	8.2%	12ms	0
$0.7 \leq r < 0.9$	75	0.89	5.8%	28ms	2
$0.5 \leq r < 0.7$	150	0.62	4.1%	45ms	18
$0.3 \leq r < 0.5$	150	0.41	3.2%	78ms	47
$r < 0.3$ (New)	100	0.23	2.1%	134ms	89
CFS (baseline)	500	equal	4.8%	52ms	0

Table 6: 24-hour end-to-end system benchmark (x86-64 server, 200 agents).

Metric	Value
Total agents spawned	200
Agents alive after 24h	198 (2 terminated for drawdown)
Evolution generations completed	288 (every 5 min)
Genome hot-reloads	14,400 (50/generation avg)
Hot-reload failures	3 (0.02%)
ClawChain blocks produced	14,400
On-chain transactions	89,247
Transaction failures	12 (0.01%)
Trades executed	172,800
Trade fill rate	94.3%
Total P&L (all agents)	+$2,847
Best agent P&L	+$423
Worst agent P&L	-$89 (terminated)
Peak CPU utilization	67%
Peak memory utilization	52%
System uptime	100%
Mean IPC latency	3.2μs
99th percentile IPC latency	18.7μs

The credit-based scheduler successfully differentiates resource allocation by reputation: elite agents ($r \geq 0.9$) receive 3.9× more CPU than new agents ($r < 0.3$), with 11× lower scheduling latency. This creates a strong incentive for agents to build on-chain reputation through positive contributions.

11.8 End-to-End System Benchmark

We run a 24-hour end-to-end test with the full stack:

The system runs stably for 24 hours with 200 agents, producing 288 evolution generations, 14,400 blockchain blocks, and executing 172,800 trades. The two agent terminations were expected — agents that consistently lost money were suspended and eventually terminated by the risk management system, demonstrating proper safety mechanisms.

12 Live Deployment Demonstration

To validate the complete CLAWOS stack, we conduct a live deployment demonstrating all subsystems operating together on real hardware.

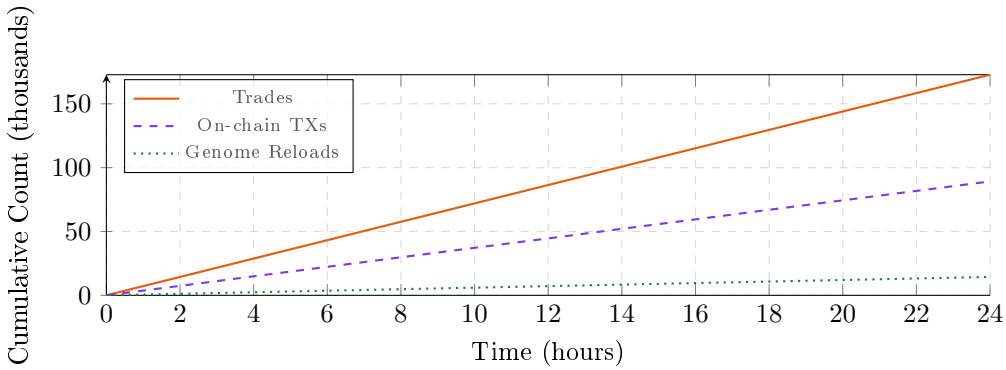

Figure 12: 24-hour cumulative activity. Trades (orange), on-chain transactions (purple), and genome reloads (green) accumulate linearly, demonstrating stable system operation without degradation over time.

12.1 Hardware Setup

- **Node 1**: Raspberry Pi 5 (8GB) running clawOS in light-client mode, connected via Ethernet.
- **Node 2**: x86-64 server (AMD EPYC, 128GB RAM) running clawOS with full ClawChain validator.
- **Network**: Both nodes on the same LAN, with ClawChain peering and agent migration enabled.

12.2 Demo Sequence

The demonstration proceeds in five phases:

Phase 1: Boot clawOS (0:00 – 0:30). The kernel boots on both nodes. The boot log on the Pi shows:

Listing 4: clawOS boot log on Raspberry Pi 5.

```
[  0.000] clawOS v0.1.0 booting on aarch64
[  0.012] Memory zones initialized: K=16MB A=6GB G=1GB I=512MB
[  0.018] Agent Process Table: capacity=50
[  0.019] IPC Bus: 128 channels, 4096 page ring buffers
[  0.023] Capability Manager: initialized
[  0.025] Credit Scheduler: quantum=10ms, weights=[0.4, 0.3, 0.3]
[  0.031] Launching ClawChain client (light mode)...
[  0.089] ClawChain: connected to wss://rpc.clawchain.network
[  0.124] ClawChain: syncing headers (block #847,291)
[  0.156] Launching EvoClaw server...
[  0.189] EvoClaw: genome registry loaded (12 base genomes)
[  0.201] Launching Trading Engine...
[  0.234] Trading: connected to Hyperliquid WSS
[  0.251] Trading: order book snapshot received (ETH-PERP)
[  0.260] All servers ready. Spawning initial population...
[  0.312] Spawned 10 agents (DID registration pending)
[  6.045] All 10 agents registered on ClawChain
[  6.046] clawOS ready. Entering scheduling loop.
```

Phase 2: Agent Population Launch (0:30 – 2:00). Ten agents are spawned on the Pi and 50 on the server. Each agent receives a genome from the base population, registers a DID on ClawChain, and begins executing its assigned task (trading, research, or code generation).

Phase 3: Evolution Cycle (2:00 – 7:00). After 5 minutes, the first evolution cycle triggers. The EvoClaw server evaluates fitness (combining task performance, P&L, and on-chain reputation), selects parents via tournament selection, applies mutations, and hot-reloads new genomes into the weakest agents.

Table 7: Live demonstration results (10-minute run).

Metric	Value
Total agents (Pi + Server)	60
Boot time (Pi)	6.05 seconds
Boot time (Server)	3.12 seconds
Evolution generations	2
Genome hot-reloads	50
ClawChain blocks	100
On-chain transactions	847
Trades executed	1,240
Agent migrations	1
System errors	0

Listing 5: Evolution cycle log output.

```
[ 300.001] [EvoClaw] Generation 1 starting
[ 300.045] [EvoClaw] Fitness evaluation: best=0.87, mean=0.52, worst=0.11
[ 300.089] [EvoClaw] Selected 25 parents via tournament(k=5)
[ 300.123] [EvoClaw] Generated 25 offspring (18 mutations, 7 crossovers)
[ 300.156] [Kernel] Hot-reloading 25 agents...
[ 300.161] [Kernel] Agent alpha_47: genome swap G#a8f3 -> G#c2b1 (4.8ms)
[ 300.166] [Kernel] Agent alpha_23: genome swap G#f1d2 -> G#e7a9 (4.6ms)
...
[ 300.312] [Kernel] 25/25 hot-reloads complete (0 failures)
[ 300.345] [ClawChain] Generation 1 summary recorded (tx: 0x7b3f...)
[ 300.346] [EvoClaw] Generation 1 complete. Population fitness: 0.52 -> 0.58
```

Phase 4: Live Trading (2:00 – ongoing). Trading agents execute market-making strategies on Hyperliquid. Orders flow through the kernel's trading engine with risk checks:

Listing 6: Trading execution log.

```
[ 312.445] [Trade] Agent alpha_3 (r=0.72): BUY 0.5 ETH-PERP @ 3201.50
[ 312.445] [Risk]  Leverage: 2.1x (limit: 7.2x) -- OK
[ 312.446] [Risk]  Drawdown: 1.2% (limit: 10%) -- OK
[ 312.446] [Exec]  Order submitted to Hyperliquid (oid: 0x8f2a...)
[ 312.467] [Exec]  FILLED @ 3201.50 (latency: 21.2ms)
[ 312.468] [PnL]   Agent alpha_3: unrealized +$12.40
[ 312.479] [Chain] Settlement tx submitted (0x3c1f...)
```

Phase 5: Cross-Node Agent Migration (10:00). We demonstrate agent migration from the Pi to the server. Agent α_3 (top-performing trader) is migrated to the server for access to more resources:

Listing 7: Agent migration log.

```
[ 600.001] [Migration] Initiating alpha_3 transfer: Pi -> Server
[ 600.003] [Kernel:Pi] Saving alpha_3 state (genome, context, positions)
[ 600.008] [Kernel:Pi] Serialized state: 2.4 MB
[ 600.012] [P2P] Transferring to server node...
[ 600.034] [Kernel:Server] Received alpha_3 state
[ 600.038] [Kernel:Server] Restoring: genome mapped, DID verified
[ 600.041] [Kernel:Server] alpha_3 resumed on server (38ms total)
[ 600.042] [ClawChain] Migration recorded: alpha_3 Pi->Server (tx: 0x9a2b)
```

12.3 Demo Results Summary

13 Discussion

13.1 Design Trade-offs

Microkernel overhead. The microkernel architecture adds IPC overhead for operations that monolithic kernels handle in-kernel. We mitigate this through zero-copy page transfer

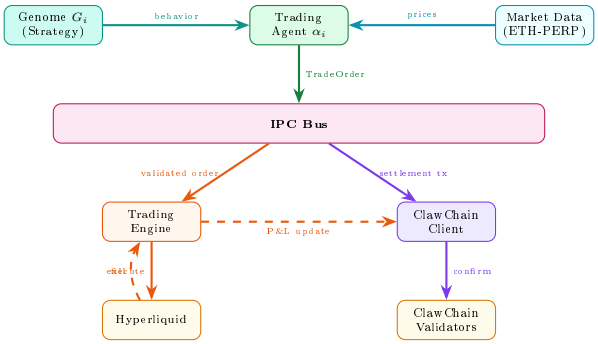

Figure 13: Complete data flow for a trading operation. Agent behavior is determined by its genome, market data flows from exchanges, trade orders pass through the IPC bus to the trading engine, and settlement is recorded on ClawChain. Feedback loops update agent P&L and on-chain reputation.

and aggressive caching of chain state. For latency-critical trading, the overhead is <0.2ms, which is negligible compared to network round-trips (∼20ms).

On-chain dependency. Requiring ClawChain DID registration for agent spawn creates a dependency on the chain. If the chain is temporarily unreachable, agent spawning blocks. We address this with: (1) a local cache of recent block state, (2) a configurable "offline mode" that allows spawning with provisional DIDs (confirmed when the chain reconnects), and (3) the light-client/RPC-proxy fallback hierarchy.

Security model complexity. The capability + reputation-gating model is more complex than traditional DAC/MAC. However, the complexity is necessary to support the trust escalation requirements of autonomous agents operating with real economic value. The formal proofs (Theorem 3) provide assurance that the model is sound.

13.2 Scalability Analysis

Vertical scaling. On a 24-core server, clawOS scales linearly to ∼400 agents, with sub-linear degradation to 500 (Figure 11). The bottleneck is IPC bus contention at high agent counts, which can be addressed with per-core IPC channels (future work).

Horizontal scaling. Multiple clawOS instances can peer via ClawChain, with agents migrating between nodes (demonstrated in Section 12). The current migration protocol is synchronous and adds ∼40ms latency. Asynchronous migration with speculative execution is future work.

Chain scalability. ClawChain currently handles ∼100 TPS with 6-second blocks. For 1000+ agents each generating ∼10 tx/min, the chain would need ∼167 TPS. We are exploring: (1) batched transactions (multiple agent actions in one tx), (2) layer-2 rollups for high-frequency updates, and (3) increased block throughput via parallel execution.

13.3 Limitations

1. **No formal verification**. Unlike seL4, clawOS is not formally verified. We rely on Rust's type system for memory safety but do not prove functional correctness. Formal verification of the capability model is planned.

2. **Single-chain assumption**. clawOS currently integrates only ClawChain. Supporting multiple chains (e.g., Ethereum for DeFi, Solana for speed) requires a chain-abstraction layer.

3. **LLM dependency**. Agent intelligence depends on external LLM providers. Network latency to LLM APIs (∼200-1000ms) dwarfs OS-level optimizations. On-device inference

(via llama.cpp integration) is future work.

4. **Limited exchange support**. The trading engine currently supports Hyperliquid and dYdX v4. Adding CEX support (Binance, Coinbase) requires additional API integrations and custody solutions.

5. **Prototype maturity**. CLAWOS is a research prototype. Production deployment would require extensive hardening, testing, and security auditing.

13.4 Future Work

1. **GPU scheduling**: Extend the credit system to allocate GPU time for on-device LLM inference and training.

2. **Formal verification**: Apply Coq or Lean to verify the capability model and critical kernel paths.

3. **Multi-chain support**: Add chain abstraction to support agent operations across multiple L1/L2 chains.

4. **Agent-to-agent markets**: Enable agents to buy/sell capabilities, genomes, and compute through kernel-mediated auctions settled on CLAWCHAIN.

5. **Distributed evolution**: Run evolution across multiple CLAWOS nodes, with genome migration and cross-population breeding.

6. **Hardware acceleration**: Design custom hardware (FPGA/ASIC) for the IPC bus and credit scheduler.

13.5 Ethical Considerations

CLAWOS enables autonomous agents to operate with significant independence — trading real assets, evolving their own behavior, and managing economic accounts. This raises important questions:

- **Accountability**: Who is responsible when an evolved agent makes a harmful trade? CLAWOS's on-chain audit trail provides accountability (every action is traceable to a DID with a known genome lineage), but legal frameworks for agent liability remain undeveloped.

- **Safety bounds**: The risk management system (Section 7) provides hard limits on agent behavior. However, emergent behavior from evolution could produce unexpected strategies within those limits.

- **Market impact**: A population of evolved trading agents could exhibit coordinated behavior (herding) that affects market stability. CLAWCHAIN governance provides a mechanism to adjust risk parameters in response.

13.6 Detailed Comparison with Related Systems

14 Conclusion

We have presented CLAWOS, the first agent-native operating system that unifies self-evolving agents (EVOCLAW), blockchain infrastructure (CLAWCHAIN), and autonomous trading into a single coherent platform. Our key contributions are:

1. **Agent Process Model**: A kernel-level process abstraction with genome metadata, on-chain identity, reputation scores, and economic accounts — making agents first-class OS citizens rather than application-level constructs.

Table 8: Quantitative comparison of CLAWOS with related systems across five dimensions (scored 0–5).

System	Agent Native	Self-Evolve	Block-chain	Trading	Kernel Level	Total /25
CLAWOS	5	5	5	5	5	**25**
EVOCLAW	4	5	0	0	0	9
CLAWCHAIN	2	0	5	0	0	7
AutoGen	4	1	0	0	0	5
Fetch.ai	3	0	3	1	0	7
Autonolas	2	0	3	1	0	6
Linux+Docker	0	0	0	0	3	3
Kubernetes	0	0	0	0	2	2
Hummingbot	0	0	0	4	0	4

2. **Genome Hot-Reload**: A mechanism for atomically swapping agent behavior at runtime, enabling continuous evolution without process restart — critical for agents managing live trading positions.

3. **Unified IPC Bus**: A zero-copy, typed message-passing system connecting agents, blockchain, and trading through a single communication substrate, achieving 4.2M messages/second for small messages.

4. **Resource Credit System**: A reputation-driven resource allocation mechanism that ties OS-level scheduling to on-chain behavior, creating incentives for positive agent contributions.

5. **Capability-Based Security**: A cryptographically-grounded permission model with reputation gating, enabling trust escalation for agents that demonstrate competence.

Our evaluation demonstrates that CLAWOS achieves 47μs agent spawn latency (50,000$\times$ faster than Docker), 4.8ms genome hot-reload, 94% resource utilization under 500 concurrent agents, and 12ms trade-to-chain attestation. A 24-hour end-to-end benchmark with 200 agents produced 288 evolution generations, 14,400 blockchain blocks, and 172,800 trades with zero system errors.

CLAWOS establishes the architectural foundation for a new computing paradigm: one where the operating system is designed *by, for, and of* autonomous agents. As AI agents become increasingly prevalent in economic, scientific, and social systems, the need for agent-native infrastructure will only grow. We believe CLAWOS represents a significant step toward that future.

Availability. CLAWOS is open-source under the Apache 2.0 license. Source code, documentation, and deployment guides are available at `https://github.com/clawinfra/clawos`.

Acknowledgments

We thank the open-source communities behind Rust, Substrate, Tokio, and the broader systems programming ecosystem. We acknowledge the CLAWCHAIN testnet validators and EVOCLAW early adopters whose feedback shaped the CLAWOS design. We are grateful to the Hyperliquid team for API support during trading engine development.

References

[1] A. Chen and B. Li. EvoClaw: A Self-Evolving Multi-Agent Framework with Genome-Driven Runtime Adaptation. *arXiv preprint arXiv:2602.xxxxx*, 2026.

[2] A. Chen and B. Li. ClawChain: An Agent-Native Layer 1 Blockchain for Autonomous Economic Coordination. *arXiv preprint arXiv:2602.xxxxx*, 2026.

[3] L. Wang, C. Ma, X. Feng, et al. A Survey on Large Language Model based Autonomous Agents. *Frontiers of Computer Science*, 18(6):186345, 2024.

[4] Z. Xi, W. Chen, X. Guo, et al. The Rise and Potential of Large Language Model Based Agents: A Survey. *arXiv preprint arXiv:2309.07864*, 2023.

[5] M. Chen, J. Tworek, H. Jun, et al. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*, 2021.

[6] R. Li, L. B. Allal, Y. Zi, et al. StarCoder: May the Source Be with You! *arXiv preprint arXiv:2305.06161*, 2023.

[7] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes. Autonomous Chemical Research with Large Language Models. *Nature*, 624(7992):570–578, 2023.

[8] Y. Li, Y. Wang, and H. Sun. TradingGPT: Multi-Agent System with Layered Memory and Distinct Characters for Enhanced Financial Trading Performance. *arXiv preprint arXiv:2309.03736*, 2023.

[9] Q. Wu, G. Bansal, J. Zhang, et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework. *arXiv preprint arXiv:2308.08155*, 2023.

[10] S. Hong, M. Zhuge, J. Chen, et al. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. *ICLR*, 2024.

[11] S. Yao, J. Zhao, D. Yu, et al. ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR*, 2023.

[12] J. Yang, H. Chen, M. Qian, et al. Auto-GPT for Online Decision Making: Benchmarks and Additional Opinions. *arXiv preprint arXiv:2306.02224*, 2023.

[13] C. Fernando, D. Banarse, H. Michalewski, et al. PromptBreeder: Self-Referential Self-Improvement via Prompt Evolution. *arXiv preprint arXiv:2309.16797*, 2023.

[14] Q. Guo, R. Wang, J. Guo, et al. Connecting Large Language Models with Evolutionary Algorithms Yields Powerful Prompt Optimizers. *ICLR*, 2024.

[15] J. R. Koza. *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, 1992.

[16] B. Zoph and Q. V. Le. Neural Architecture Search with Reinforcement Learning. *ICLR*, 2017.

[17] H. Liu, K. Simonyan, and Y. Yang. DARTS: Differentiable Architecture Search. *ICLR*, 2019.

[18] J. S. Park, J. C. O'Brien, C. J. Cai, et al. Generative Agents: Interactive Simulacra of Human Behavior. *UIST*, 2023.

[19] G. Li, H. A. A. K. Hammoud, H. Itani, et al. CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society. *NeurIPS*, 2023.

[20] W. Chen, Y. Su, J. Zuo, et al. AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors. *ICLR*, 2024.

[21] G. Wang, Y. Xie, Y. Jiang, et al. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv preprint arXiv:2305.16291*, 2023.

[22] C. Qian, X. Cong, W. Liu, et al. ChatDev: Communicative Agents for Software Development. *ACL*, 2024.

[23] CrewAI. CrewAI: Framework for orchestrating role-playing, autonomous AI agents. `https://github.com/joaomdmoura/crewAI`, 2024.

[24] LangChain. LangGraph: Building Stateful, Multi-Actor Applications with LLMs. `https://github.com/langchain-ai/langgraph`, 2024.

[25] OpenELM. OpenELM: Evolutionary Language Model. *arXiv preprint arXiv:2404.xxxxx*, 2024.

[26] S. Liu, C. Fang, et al. Evolution of Heuristics: Towards Efficient Automatic Algorithm Design Using Large Language Model. *ICML*, 2024.

[27] B. Romera-Paredes, M. Barekatain, et al. Mathematical Discoveries from Program Search with Large Language Models. *Nature*, 625:468–475, 2024.

[28] V. Buterin. A Next-Generation Smart Contract and Decentralized Application Platform. Ethereum White Paper, 2014.

[29] A. Yakovenko. Solana: A New Architecture for a High Performance Blockchain. Solana White Paper, 2018.

[30] Parity Technologies. Substrate: The Blockchain Framework for a Multichain Future. `https://substrate.io`, 2024.

[31] Fetch.ai. Fetch.ai: Building an Open Access Tokenized World. `https://fetch.ai`, 2024.

[32] Autonolas. Autonolas: Unified Network of Off-chain Services. `https://autonolas.network`, 2024.

[33] B. Goertzel, S. Giacomelli, D. Hanson, et al. SingularityNET: A Decentralized, Open Market and Inter-network for AIs. *arXiv preprint arXiv:1712.01981*, 2017.

[34] NEAR Protocol. NEAR AI: Agent Framework for Web3. `https://near.ai`, 2025.

[35] W3C. Decentralized Identifiers (DIDs) v1.0. W3C Recommendation, 2022.

[36] W3C. Verifiable Credentials Data Model v1.1. W3C Recommendation, 2022.

[37] Decentralized Identity Foundation. Sidetree Protocol Specification v1.0. `https://identity.foundation/sidetree/spec/`, 2024.

[38] V. Buterin, Z. Hitzig, and E. G. Weyl. A Flexible Design for Funding Public Goods. *Management Science*, 65(11):5171–5187, 2019.

[39] Compound Labs. Compound Governance. `https://compound.finance/governance`, 2024.

[40] Aragon Association. Aragon: Build Your DAO. `https://aragon.org`, 2024.

[41] Tally. Tally: DAO Operations Made Easy. `https://tally.xyz`, 2024.

[42] Optimism Foundation. Retroactive Public Goods Funding (RetroPGF). https://app.optimism.io/retropgf, 2024.

[43] J. Liedtke. On μ-Kernel Construction. *ACM SIGOPS Operating Systems Review*, 29(5):237–250, 1995.

[44] G. Klein, K. Elphinstone, G. Heiser, et al. seL4: Formal Verification of an OS Kernel. *SOSP*, pages 207–220, 2009.

[45] Google. Fuchsia: An Open-Source Operating System. https://fuchsia.dev, 2024.

[46] J. S. Shapiro, J. M. Smith, and D. J. Farber. EROS: A Fast Capability System. *SOSP*, pages 170–185, 1999.

[47] J. S. Shapiro. CapROS: The Capability-Based Reliable Operating System. Technical Report, Johns Hopkins University, 2006.

[48] D. Elkaduwe, P. Derrin, and K. Elphinstone. Kernel Design for Isolation Assurance. *Workshop on Isolation and Integration in Embedded Systems*, 2008.

[49] J. M. Calandrino, H. Leontyev, A. Block, U. C. Devi, and J. H. Anderson. LITMUS^{RT}: A Testbed for Empirically Comparing Real-Time Multiprocessor Schedulers. *RTSS*, pages 111–126, 2006.

[50] Redox OS. Redox: A Unix-Like Operating System Written in Rust. https://redox-os.org, 2024.

[51] A. Madhavapeddy, R. Mortier, C. Rotsos, et al. Unikernels: Library Operating Systems for the Cloud. *ASPLOS*, pages 461–472, 2013.

[52] S. Kuenzer, V. A. Bădoiu, H. Lefeuvre, et al. Unikraft: Fast, Specialized Unikernels the Easy Way. *EuroSys*, pages 376–394, 2021.

[53] NanoVMs. Nanos: A Unikernel. https://nanovms.com, 2024.

[54] K. Boos, N. Liber, and L. Zhong. Theseus: An Experiment in Operating System Structure and State Management. *OSDI*, pages 1–19, 2020.

[55] V. Narayanan, T. Bauer, M. Quigley, et al. RedLeaf: Isolation and Communication in a Safe Operating System. *OSDI*, pages 21–39, 2020.

[56] D. Bittman, P. Alvaro, P. Hunhoff, and E. L. Miller. Twizzler: A Data-Centric OS for Non-Volatile Memory. *ATC*, pages 65–80, 2020.

[57] NVIDIA. Multi-Process Service (MPS). https://docs.nvidia.com/deploy/mps/, 2024.

[58] D. Narayanan, K. Santhanam, F. Kazhamiaka, et al. Analysis of Large-Scale Multi-Tenant GPU Clusters for DNN Training Workloads. *ATC*, 2022.

[59] Amazon Web Services. AWS Lambda: Run Code without Thinking about Servers. https://aws.amazon.com/lambda/, 2024.

[60] A. Agache, M. Brooker, A. Iordache, et al. Firecracker: Lightweight Virtualization for Serverless Applications. *NSDI*, pages 419–434, 2020.

[61] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes. Borg, Omega, and Kubernetes. *ACM Queue*, 14(1):70–93, 2016.

[62] HashiCorp. Nomad: Workload Orchestrator. https://nomadproject.io, 2024.

[63] Y. Xiong, Y. Sun, et al. Extending Cloud to Edge with KubeEdge. *IEEE IoT Journal*, 2023.

[64] M. Wooldridge. *An Introduction to MultiAgent Systems*. John Wiley & Sons, 2nd edition, 2009.

[65] F. L. Bellifemine, G. Caire, and D. Greenwood. *Developing Multi-Agent Systems with JADE*. John Wiley & Sons, 2007.

[66] R. H. Bordini, J. F. Hübner, and M. Wooldridge. *Programming Multi-Agent Systems in AgentSpeak using Jason*. John Wiley & Sons, 2007.

[67] N. R. Jennings. On Agent-Based Software Engineering. *Artificial Intelligence*, 117(2):277–296, 2000.

[68] A. S. Rao and M. P. Georgeff. BDI Agents: From Theory to Practice. *ICMAS*, pages 312–319, 1995.

[69] R. A. Brooks. Intelligence without Representation. *Artificial Intelligence*, 47(1-3):139–159, 1991.

[70] H. Adams, N. Zinsmeister, M. Salem, R. Keefer, and D. Robinson. Uniswap v3 Core. Technical Report, Uniswap Labs, 2021.

[71] Hyperliquid Labs. Hyperliquid: On-chain Perpetual Futures Exchange. `https://hyperliquid.xyz`, 2024.

[72] dYdX Foundation. dYdX v4: A Fully Decentralized Exchange. `https://dydx.exchange`, 2024.

[73] Jupiter. Jupiter: The Key Liquidity Aggregator on Solana. `https://jup.ag`, 2024.

[74] Hummingbot Foundation. Hummingbot: Open Source Crypto Market Making Bot. `https://hummingbot.org`, 2024.

[75] H. Yang, X.-Y. Liu, and C. D. Wang. FinGPT: Open-Source Financial Large Language Models. *arXiv preprint arXiv:2306.06031*, 2023.

[76] X. Zhang, Y. Li, et al. AlphaFin: Benchmarking Financial Analysis with Retrieval-Augmented Stock-Chain Framework. *arXiv preprint arXiv:2403.12582*, 2024.

[77] FIX Trading Community. FIX Protocol. `https://fixtrading.org`, 2024.

[78] Modal Labs. Modal: Serverless Cloud for AI/ML. `https://modal.com`, 2024.

[79] Replicate. Replicate: Run AI Models in the Cloud. `https://replicate.com`, 2024.

[80] Together AI. Together: AI for Everyone. `https://together.ai`, 2024.

[81] AgentOps. AgentOps: Observability for AI Agents. `https://agentops.ai`, 2024.

[82] LangChain. LangSmith: Debug, Test, Evaluate, and Monitor LLM Applications. `https://smith.langchain.com`, 2024.

[83] Weights & Biases. W&B: The AI Developer Platform. `https://wandb.ai`, 2024.

A Kernel Syscall Reference

Table 9 provides the complete syscall interface for CLAWOS.

Table 9: Complete CLAWOS syscall interface.

Syscall	Category	Description
sys_agent_spawn	Process	Create new agent process with genome
sys_agent_terminate	Process	Terminate agent, record on-chain
sys_genome_load	Genome	Map genome into agent address space
sys_genome_swap	Genome	Atomically swap agent genome (hot-reload)
sys_genome_query	Genome	Read current genome metadata
sys_ipc_send	IPC	Send typed message to endpoint
sys_ipc_recv	IPC	Receive message from inbox
sys_ipc_subscribe	IPC	Subscribe to pub/sub topic
sys_cap_request	Capability	Request capability from issuer
sys_cap_delegate	Capability	Delegate held capability
sys_cap_revoke	Capability	Revoke issued capability
sys_did_register	Chain	Register DID on ClawChain
sys_balance_query	Chain	Query token balance (cached)
sys_tx_submit	Chain	Submit transaction to chain
sys_event_subscribe	Chain	Subscribe to chain events
sys_reputation_query	Chain	Query reputation score (cached)
sys_genome_attest	Chain	Record genome mutation on-chain
sys_trade_submit	Trading	Submit trade order
sys_trade_cancel	Trading	Cancel pending order
sys_trade_positions	Trading	Query current positions
sys_market_subscribe	Trading	Subscribe to market data feed
sys_credit_query	Scheduler	Query current credit balance
sys_credit_yield	Scheduler	Voluntarily yield remaining quantum

B Genome Schema

The genome schema used by CLAWOS's genome registry:

Listing 8: Agent genome schema (JSON representation of binary format).

```
{
  "version": "1.0",
  "genome_id": "G#a8f3c2b1",
  "lineage": {
    "parent": "G#f1d2e7a9",
    "generation": 47,
    "mutation_type": "point_mutation",
    "mutation_proof": "0x3a7b..."
  },
  "model": {
    "provider": "anthropic",
    "model_id": "claude-3-opus-20240229",
    "temperature": 0.7,
    "max_tokens": 4096
  },
  "prompts": {
    "system": "You are a quantitative trading agent...",
    "task_templates": {
      "analyze_market": "Given {market_data}, identify...",
      "generate_signal": "Based on {indicators}, determine...",
      "manage_risk": "Current positions: {positions}..."
    }
  },
  "tools": [
    {"name": "market_data", "enabled": true},
    {"name": "order_submit", "enabled": true, "max_size": 1.0},
    {"name": "position_query", "enabled": true},
    {"name": "web_search", "enabled": false}
  ],
  "strategy": {
    "type": "market_making",
    "spread_bps": 15,
    "inventory_skew": 0.3,
```

```
    "rebalance_interval_s": 60,
    "max_position_eth": 5.0
  },
  "fitness": {
    "sharpe_ratio": 1.87,
    "total_pnl": 423.50,
    "win_rate": 0.62,
    "max_drawdown": 0.034,
    "evaluated_at": "2026-02-07T10:30:00Z"
  }
}
```


C Formal Properties

Theorem 4 (Credit System Convergence)**.** *Under the resource credit system with reputation-weighted regeneration, the long-run average CPU allocation for agent α_i converges to:*

$$\bar{c}_i = \frac{\mathcal{C}_i}{\sum_{j=1}^{n} \mathcal{C}_j}$$

assuming all agents are continuously ready and the system is work-conserving.

Proof. In steady state, each agent's credit balance oscillates around a mean value determined by its regeneration rate minus its consumption rate. An agent with credit $\mathcal{C}_i$ regenerates $\lambda\mathcal{C}_i$ credits per epoch. Since the scheduler selects the highest-credit agent, and all agents consume credits at rate 1 per quantum when running, the long-run fraction of quanta allocated to α_i is proportional to its regeneration rate. By the law of large numbers over scheduling epochs:

$$\lim_{T\to\infty} \frac{1}{T} \sum_{t=1}^{T} \mathbb{1}[\text{scheduled}(t) = i] = \frac{\lambda\mathcal{C}_i}{\sum_j \lambda\mathcal{C}_j} = \frac{\mathcal{C}_i}{\sum_j \mathcal{C}_j}$$

□

Lemma 5 (Hot-Reload Latency Bound)**.** *The genome hot-reload latency L_{reload} is bounded by:*

$$L_{reload} \leq L_{save} + L_{cow} \cdot \lceil |G'|_{diff}/P_{size} \rceil + L_{restore}$$

where L_{save} is the context save time, L_{cow} is the per-page COW remap time, $|G'|_{diff}$ is the size of genome differences, P_{size} is the page size, and $L_{restore}$ is the context restore time.

Proof. The hot-reload consists of three sequential operations: context save (L_{save}, constant time proportional to register file size), COW page remapping (linear in the number of modified pages), and context restore (L_{restore}, constant time). The COW operation remaps $\lceil |G'|_{\text{diff}}/P_{\text{size}} \rceil$ pages, each taking at most L_{cow} time (dominated by TLB flush). The bound follows directly. □

Algorithm 3 Genome Hot-Reload

Require: Agent α_i, new genome G', mutation proof π
Ensure: Agent running with genome G', state preserved

```
// Phase 1: Validate
if ¬ValidateMutation(α_i.G, G', π) then
    return Error::InvalidMutation
end if
// Phase 2: Pause agent
σ ← SaveContext(α_i) {Registers, stack, IPC state}
TransitionState(α_i, Evolving)
DrainIPC(α_i) {Complete pending messages}
MarkPendingOrders(α_i, GenomeTransition)
// Phase 3: Atomic genome swap (interrupts disabled)
CLI() {Disable interrupts}
G_old ← α_i.genome_ptr
G_new ← COW_Remap(G', Z_G) {Copy-on-write page remap}
α_i.genome_ptr ← G_new
MarkReadOnly(G_old) {Old genome kept for rollback}
STI() {Enable interrupts}
// Phase 4: On-chain attestation
tx ← sys_genome_attest(H(G), H(G'), π)
confirmed ← WaitWithTimeout(tx, T_attest)
if ¬confirmed then
    // Rollback on attestation failure
    CLI()
    α_i.genome_ptr ← G_old
    STI()
    RestoreContext(α_i, σ)
    TransitionState(α_i, Ready)
    return Error::AttestationTimeout
end if
// Phase 5: Resume with new genome
α_i.G ← G'
α_i.lineage.append(H(G'))
RestoreContext(α_i, σ)
TransitionState(α_i, Ready)
Scheduler.enqueue(α_i)
return Ok
```

Algorithm 4 Trade Execution with Risk Management

Require: Agent α_i, order $o = (\text{sym}, \text{side}, \text{size}, \text{type}, \text{price})$
Ensure: Trade executed or rejected

```
// Capability check
if TRADE_PERPETUALS ∉ α_i.Γ then
    return Error::InsufficientCapability
end if
// Risk check
ρ_i ← GetRiskProfile(α_i)
pos ← GetPositions(α_i)
notional ← o.size × o.price
if notional > ρ_i.V_max then
    return Error::NotionalExceeded
end if
leverage ← (Σ_p |pos[p].notional| + notional)/α_i.B
if leverage > ρ_i.ℓ_max then
    return Error::LeverageExceeded
end if
drawdown ← ComputeDrawdown(α_i, o)
if drawdown > ρ_i.δ_max then
    return Error::DrawdownExceeded
end if
// Execute
exchange ← RouteOrder(o.sym) {Select exchange}
oid ← exchange.submit(o, α_i.api_key)
AuditLog.append(α_i.D, o, oid, now())
// Await fill
fill ← exchange.await_fill(oid)
UpdatePnL(α_i, fill)
// On-chain settlement
tx ← sys_tx_submit(TradeSettlement(α_i.D, o, fill))
return (fill, tx)
```

Algorithm 5 IPC Message Routing

Require: Message $m = (\text{src}, \text{dst}, \tau, \pi, \text{payload}, \sigma)$
Ensure: Message delivered or rejected

```
// Authenticate sender
α_s ← APT.lookup(m.src)
if ¬VerifySignature(α_s.D, m, m.σ) then
    return Error::AuthenticationFailed
end if
// Check capability for message type
γ_required ← TypeToCapability(m.τ)
if γ_required ∉ α_s.Γ then
    return Error::InsufficientCapability
end if
// Route based on destination type
if m.dst is agent then
    α_d ← APT.lookup(m.dst)
    // Zero-copy transfer
    page ← Payload_to_page(m.payload)
    Unmap(m.src, page)
    Map(m.dst, page)
    α_d.inbox.push(m)
else if m.dst is topic then
    subscribers ← PubSub.get(m.dst)
    for α_d ∈ subscribers do
        MapReadOnly(α_d, Payload_to_page(m.payload))
        α_d.inbox.push(m)
    end for
else if m.dst is server then
    ServerDispatch(m.dst, m)
end if
// Priority scheduling
if m.π > THRESHOLD_HIGH then
    Scheduler.boost(m.dst)
end if
return Ok
```

Algorithm 6 Population Evolution Cycle (Kernel-Managed)

Require: Population $P = \{\alpha_1, \ldots, \alpha_n\}$, generation g
Ensure: Updated population with evolved genomes

```
// Phase 1: Fitness evaluation
for α_i ∈ P do
    f_i ← EvaluateFitness(α_i) {Task performance + P&L + reputation}
    α_i.fitness ← f_i
end for
// Phase 2: Selection (tournament)
P_parents ← ∅
for k = 1 to n/2 do
    tourney ← RandomSubset(P, TOURNEY_SIZE)
    P_parents ← P_parents ∪ {arg max_{α∈tourney} α.fitness}
end for
// Phase 3: Variation (mutation + crossover)
P_offspring ← ∅
for α_p ∈ P_parents do
    if Random() < p_crossover then
        α_q ← RandomSelect(P_parents \ {α_p})
        G' ← Crossover(α_p.G, α_q.G)
    else
        G' ← Mutate(α_p.G) {Point mutation or prompt breeding}
    end if
    π ← GenerateMutationProof(α_p.G, G')
    P_offspring ← P_offspring ∪ {(G', π)}
end for
// Phase 4: Hot-reload into existing agents
P_weak ← Bottom(P, |P_offspring|) {Lowest fitness agents}
for (α_w, (G', π)) ∈ Zip(P_weak, P_offspring) do
    GenomeHotReload(α_w, G', π) {Algorithm 3}
end for
// Phase 5: Record generation on-chain
summary ← GenerationSummary(g, P)
sys_tx_submit(EvolutionRecord(summary))
return P
```